
TIBER CAD

StandAlone Manual

Release 2.0

TiberLAB srl

April 1, 2011

CONTENTS

I	Getting Started	1
1	Installation instructions	3
1.1	Prerequisites	3
1.2	Windows installation procedure	3
1.3	Linux installation procedure	4
1.4	Quick start guide	4
1.5	Bug reports / Feedback	5
2	Input for TiberCAD	7
2.1	Description of Input file structure	7
2.2	Device section	8
2.3	Modules	10
2.4	Simulation section	14
2.5	Output description	15
2.6	Example of Input file	16
3	Elasticity	21
3.1	Abstract	21
3.2	Mini theoretical intro	21
3.3	Example	21
4	Drift Diffusion	25
4.1	Step 1 - Modeling the device	25
4.2	Step 2 - Meshing the device	27
4.3	Step 3 - TiberCAD Input file	28
4.4	Step 4 - Run TiberCAD	30
4.5	Output	31
5	Thermal	33
5.1	Introduction	33
5.2	Boundary conditions	34
5.3	Heat sources	35

6	Quantum EFA calculations	37
6.1	Module efaschroedinger	37
6.2	Module quantumdispersion	39
6.3	Module quantumdensity	40
6.4	Module opticskp	41
6.5	Module opticalspectrum	42
7	Simulation Dye Solar Cells	45
7.1	Device	47
7.2	Physics	47
7.3	Solver	48
7.4	Output	49
II	Theory	51
8	Elasticity	53
8.1	Stiffness constant	54
9	Drift-diffusion simulation of electrons and holes	57
9.1	Theory	57
9.2	Solution/Plot variables	57
9.3	Module options	57
9.4	Solver section	58
9.5	Physics section	59
9.6	Constant mobility model	62
9.7	Doping dependent mobility model	62
9.8	Schroedinger/Poisson/Drift-Diffusion calculations	68
9.9	Example	68
10	Thermal	73
10.1	Boundary conditions	74
10.2	Heat sources	74
11	Quantum EFA calculations	77
11.1	Module efaschroedinger	77
11.2	Module quantumdispersion	80
11.3	Module quantumdensity	80
11.4	Module opticskp	82
11.5	Module opticalspectrum	83
12	Simulation Dye Solar Cells	85
12.1	Module DSC	87
12.2	Generation module	88
12.3	Device	90
12.4	Sweep	91

12.5	Output	92
III	Reference Guide	95
13	Elasticity	97
14	Solvers	101
14.1	Numerical Solvers	101
14.2	Simulations	102
14.3	Selfconsistent	103
15	Thermal	105
16	Quantum EFA calculations	107
16.1	Module efaschroedinger	107
16.2	Output	110
17	Module opticalspectrum	111
18	Output	113
18.1	Module quantumdensity	114
18.2	Module opticskp	115
18.3	Module opticalspectrum	116
IV	Glossary	119
	Bibliography	175
	Index	177

Part I

Getting Started

INSTALLATION INSTRUCTIONS

In the following, VERSION denotes the version number of the TIBERCAD release you downloaded and INSTALLPATH denotes the directory where TIBERCAD gets installed. Version 2.3.0 of **GMSH** (<http://www.geuz.org/gmsh>) will be installed together with TIBERCAD. For the Linux version of GMSH you need OpenGL libraries installed on your system.

1.1 Prerequisites

Get the installer package for your OS/architecture from <http://www.tibercad.org> or by contacting support@tibercad.org. Table 1 lists the packages available for download. To run TIBERCAD you will also need a license file that you will have to copy into the installation directory of TIBERCAD.

In the Windows version, some graphical features such as graphical convergence monitors are only available if an X Window server is installed and running.

1.2 Windows installation procedure

To install TIBERCAD in Windows, run the setup program `tibercad-version_setup.exe`.

During the installation you can choose the installation directory. After finishing installation, copy your license file (*tibercad.lic*) into the

license subdirectory of the TIBERCAD

installation directory (INSTALLPATH/license), without changing its filename.

<i>installer package name</i>	<i>Target architecture</i>
tibercad-version_setup.exe	Windows 32-bit
tibercad-version_installer.bin	Linux 32-bit self-extracting installer

Table 1: Installer packages

1.3 Linux installation procedure

To install TIBERCAD in Linux, download and run the self-extracting installer `tibercad-version_installer.bin` and follow the installation instructions.

After installation, copy your license file (*tibercad.lic*) into the “license” subdirectory of the TIBERCAD installation directory (INSTALLPATH/license) without changing the filename. You can also provide the license file during installation.

The standard method to launch TIBERCAD is by means of a shell script that is installed alongside the TiberCAD executable. It takes care of setting all necessary environment variables.

If for some reason you have to run the executable directly, remember to set TIBERCADROOT to the TiberCAD installation directory (INSTALLPATH).

1.4 Quick start guide

In the “examples” subdirectory you can find several examples ready to run. They are the same as the tutorials on <http://www.tibercad.org/documentation/tutorial/list>.

1.4.1 Windows

Open Windows Explorer and go to the TIBERCAD installation directory. If you have write permission in the installation directory, you can browse to an examples directory and start the simulation by double clicking the input file, e.g. `bulk.tib` in *Step 1 - Modeling the device*. If not, copy the whole directory to a location in your personal area and run the examples from there.

If you cannot run TIBERCAD by double clicking an input file (**.tib*), then the input files are probably not correctly associated with the TIBERCAD executable. In this case, try to establish the association by right-clicking the input file, choosing `open with...`

```
>> Choose Program... >> Browse... , browsing to the TIBERCAD in-
stallation directory
```

and choosing the TIBERCAD executable, `tibercad.exe`. A directory containing the simulation results will be created with the name provided in the input file.

1.4.2 Linux

After the correct installation of TIBERCAD you should be able to run TIBERCAD from the command line using the command `tibercad`. If not, you probably have to add the `bin` subdirectory of the TIBERCAD installation directory to your `PATH` environment variable or start the TIBERCAD executable using the absolute path (`INSTALLPATH/bin/tibercad`).

Copy the directory of the example you want to run, e.g. `bulk Si`, to your home directory or any place you have write permissions for. Change to the newly created directory and run TIBERCAD by (assuming *Step 1 - Modeling the device*)

```
$ tibercad bulk.tib
```

A directory containing the simulation results will be created with the name provided in the input file.

1.5 Bug reports / Feedback

Please send bug reports, feedback or suggestions to support@tibercad.org. When submitting bug reports, please always include the full version number of TIBERCAD you are running. The full version number appears in the first line of output when running the program:

```
$ tibercad  
TiberCAD version 1.0.0-961  
Usage: tibercad <inputfile>
```


INPUT FOR TIBERCAD

Input for TIBERCAD is composed by an input file e.g. “input.tib” and a mesh file generated by a mesher software: as for now, mesh files from GMSH (*.msh, v.1 and v.2.0) and from ISE-TCAD (*.grd) are supported. Be sure that the material files are in the correct directory . To run the program, type: **tibercad** *input_file_name*

2.1 Description of Input file structure

A valid input file for TIBERCAD is a text file with the structure described in the following. In the whole input file, everything following a “#” is considered as a comment and is disregarded; blank lines can be present anywhere and are disregarded too. Input file is composed by several **blocks** :

A block is enclosed between “{” and “}” brackets and may include one or more blocks. Each block has a header made of one or two keywords. Each block may contain zero or any number of **parameter assignments** in the form:

” *tagname* = *tagvalue*” , where

- “*tagname*” is a string
- “*tagvalue*” is a single numerical or string item or a list of items between “(” and “)”

parenthesis and separated by commas. e.g. (*cathode*, *anode*)

Format is free for the parameter assignments, provided that they are separated by spaces. Everything which follows a “#” is considered as a comment and is disregarded. For example:

```
electron_mobility field_dependent
{
  region = buffer
  # type = doping_dependent
  low_field_model = constant
}
```

Here and in the whole input file a string item can include a combination of characters, special characters and numbers, except spaces; if a space is found, the string item is taken as terminated.

The input file is composed by the following main classes of blocks:

Device, Module, Simulation

which will be described in the following.

2.2 Device section

```
Device hemt1
{
  meshfile = hemt_msh.grd
  Region buffer
  {
    .....
    Doping
    {
      .....
    }
  }
  Region barrier_1
  {
    .....
  }
  .....
}
```

Device section includes the geometrical description of the device to be simulated; an optional *device name* can be associated to the Device object, e.g. *Device hemt1* .

In Device section, two kinds of block can be present: the Region block and the Cluster

block. Outside of these blocks, general options common to all the device can be defined. The most important is the mesh file definition:

meshfile : name of mesh file. N.B.: the extension is mandatory! (*.grd* for ISE-TCAD, *.msh* for GMSH mesh file v.1 and v.2.0)

The definition of the mesh file associated to this simulation is compulsory:

```
meshfile = hemt_msh.grd
```

The **Region** blocks contain the description of the device in continuous media approach; the **Cluster** blocks define each a group of regions (mesh regions) even with different physical properties, but to be treated together somewhere in the simulation (e.g. quantum calculation). In this way it is possible to refer to the set of these regions simply by the **Cluster** name.

Each **Region** block must be preceded by the keyword “**Region**” , followed by the (single-word) name of the **TiberCAD Region** . The name of the **TiberCAD Region** can coincide with the name of a mesh region, as defined during the modeling of the device; in this case, if the keyword *mesh_regions* is absent, the **TiberCAD Region** will be associated to the mesh region identified by the name assigned to the **TiberCAD Region** . Otherwise, the **TiberCAD Region** will be associated to the mesh regions specified by the keyword *mesh_regions* .

```
Region QWell
{
  mesh_regions = (well1,well2)
  structure = wz
  y-growth-direction = (1,0,-1,0)
  z-growth-direction = (-1,2,-1,0)
  x-growth-direction = (0,0,0,1)
  material = GaN
  Doping
  {
    density = 1e17
    type = donor
    level = 0.025
  }
}
```

Here are the description of the available keywords for a **Region** block.

material (mandatory): name of the material associated to the present region, e.g Si;

it may be a ternary alloy, e.g AlGaAs, in this case keyword x described in the following has to be present.

x : alloy concentration, expressed as the molar fraction of the first component of the alloy; e.g. to express an alloy $Al_xGa_{1-x}As$ with molar fraction $x = 0.2$, that is $Al_{0.2}Ga_{0.8}As$, we select **AlGaAs** for the keyword material, and 0.2 for the keyword x, thus we write $x = 0.2$.

mesh_regions : (a list of) region physical name(s) as specified in the meshing program.

structure : crystal structure (wz = wurtzite, zb = zincblend)

x-growth-direction, **y-growth-direction**,
z-growth-direction : Bravais vectors with Miller

indexes for wurtzite crystal (4 element vectors) or zincblende crystal (3 element vectors).



A common crystal structure and growth directions definition may be applied to all the Regions of the Device, just by defining them everywhere in the Device block, but outside any specific Region block.

Subblock doping, with the keywords:

density : doping concentration [cm⁻³]
type : donor or acceptor
level : energy level of the dopant [eV]

```
Doping
{
  density = 1e21
  type = donor
  level = 0.026
}
```

Each **Cluster** block must be preceded by the keyword “**Cluster**” , followed by the (single-word) name of the **Cluster** .

```
Cluster Quantum_1
{
  regions = (well, barrier1, barrier2)
}
```

regions (mandatory): list of physical regions as specified in the meshing program, or TIBERCAD region names to be grouped in the cluster.

Regions and **Clusters** represent the macroscopical description of the device or structure

to be simulated in **TiberCAD** . In the rest of the input file, the physical regions associated to the **Modules** will be indicated by means of the **TiberCAD Region** and **Cluster** names.

2.3 Modules

```
Module driftdiffusion
{
  name = dd
}
```



```

#regions = all
statistics = FermiDirac
strain_simulation = macrostrain
plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)
.....
Solver
{
  nonlinear_solver = tiber
  nonlin_max_it = 15
  nonlin_rel_tol = 1e-12
  #nonlin_abs_tol = 1e-15
  nonlin_step_tol = 1e-5
}
Physics
{
  polarization (piezo, pyro) {}
  mobility
  {
    type = field_dependent
    low_field_model = doping_dependent
  }
}
Contact cathode
{
  voltage = $Vd
}
Contact anode
{
  voltage = 0.0
}
}

```

One or more module-blocks may be present: each module-block must be preceded by the keyword **“Module”**, followed by the (single-word) module name. This must be the name of one of the TIBERCAD modules. Here are the Modules implemented until now:

- `driftdiffusion` : Poisson-driftdiffusion transport of electrons and holes
- `thermal` : Heat balance simulation
- `excitontransport` : Exciton transport model
- `macrostrain` : Calculation of Elastic deformations in heterostructures
- `efaschroedinger` : Envelop Function Approximation (EFA) solution of single particle

Schroedinger equation for electrons and holes

- `quantumdensity` : Calculation of quantum density of electrons and holes.
- `quantumdispersion` : Dispersion of quantized states in k space
- `opticskp` : Optical properties (optical kp matrix elements)
- `opticalspectrum` : Emission spectrum (with k-space integration)
- `Sweep` : Parameterized execution of a module simulation (e.g. for the calculation

of output current characteristics)

- `Selfconsistent` : coupled calculations of different simulation modules.
- `DSC` : Simulation of a DSC solar cell1.

Each module-block usually contains a list of general options, such as plot and others specific to each module. Then, two main blocks define the Physics and the Solver models and parameters for this module.

Solver contains the solver parameters; depending on the Module, it can contain a `LinearSolver` definition subblock.

Physics usually contains the definition of the physical models used in the simulation.

For example, for `driftdiffusion` module, recombination, electron mobility, trap, polarization and so on. A particular model is the Boundary model, which has an alias `Contact` for `driftdiffusion` module. The declaration of these models obey to the following syntax:

model_keyword type_specifier <block>

where *model_keyword* is the name of the physical model to be declared (e.g. trap model) and **type_specifier** is the name of a particular one among the available descriptions for that model. For example, for a trap model of **type** *acceptor*

```
trap acceptor
{
    region = buffer
    Nt = 7e16
    Et = 0.5
    reference = cb
}
```

An alternative multiple declaration is possible if no parameters, beyond default, are declared:

```
model_keyword (type specifier1, type specifier2,...)

recombination (srh, direct, auger) { }
```

In this example, several recombination models are defined (srh, auger, direct) each one with default parameters. For a detailed description of the models, please refer to the reference guide. Two special Modules are: the Sweep Module and the Selfconsistent Module

2.3.1 Module sweep

Module sweep

```
{
  name = sweep_drain
  solve = driftdiffusion
  variable = $Vd
  start = 0.0
  stop = 2.0
  steps = 20
  plot_data = true
}
```

Module sweep

```
{
  name = sweep_gate
  solve = sweep_drain
  variable = $Vg
  start = -0.1
  stop = 1.0
  steps = 11
}
```

Each sweep Module defines a set of calculations applied to a boundary region (e.g. a set of bias values to be assigned to a drain contact of a MOSFET for the calculation of an output drain IV characteristic), in this Guide referred to as sweep calculation.

The following keywords are defined for this feature:

`variable` : name of the variable to which the sweep is applied:

E.g.: `variable = $Vg`

indicates that the values are applied to the variable Vg, which is a quantity defined in a **Contact** definition

`start, stop, steps` : sweep starts from start value, is repeated steps times and stops

in stop

`solve` : name of the simulation (module) associated to the sweep calculation; it may

be the name of another sweep defined in the same block.

`plotvariable` (obsolete): specify the integrated quantity to be calculated during the

sweep and that will be shown in the output file `sweep_modelname_sweepvariable.dat`, eg. `sweep driftdiffusion Vb.dat` for a sweep of current calculation on the variable Vb (typically a contact voltage).

`plot_data` : default is false; if it is set to true, then output data will be written for each step of the sweep calculation, otherwise just the results for the final step will be present in the output.

Once a *sweep* calculation has been defined, it is treated as a special case of simulation and may be executed as an usual simulation: by adding it in the solve list, e.g. `solve = sweep_drain`.

2.3.2 Module selfconsistent

In this Module it is possible to define a self-consistent calculation based on two different simulation modules (e.g. `driftdiffusion` and `excitontransport`).

```
Module selfconsistent
{
  name = sc_all
  solve = (tb, dens_el, dens_hl, dd)
  max_iterations = 5
  abs_tolerance = 1e-3
  rel_tolerance = 1e-6
}
```

In **solve** the list of simulations to be performed self-consistently is specified. Now it is possible to execute the specified simulations in self consistent way, by using the **selfconsistent** keyword like a simulation name, in any **solve** assignment, e.g. `solve = selfconsistent` in **Simulation** section, or even in a **sweep** section.

2.4 Simulation section

In this block one can specify several general parameters and settings for the actual calculation to be run, such as the temperature, the process-flow of simulation, etc.

`searchpath` : path for material files

`mesh units` : units of measurements used in the meshing (relative to meters): e.g.,
 10^{-6} for μm

`dimension` : dimension of simulation (1,2,3)

`temperature` : temperature of the system [K]

`solve` : list of simulations to be executed, in the order of execution; if the list contains

“sweep”, a sweep is performed as specified in sweep block in the Solver section.

```
solve = (strain,driftdiffusion,
quantum_electrons, quantum_holes)
```

`resultpath` : path for output directory

`output format` : format of the output data: *gmV* for **GMV** , *ise* for **Tecplot** , *grace* for

xmgr (ascii data column type), *vtk* for **Paraview** .

`plot` : list of output variables which are calculated and available in output files. It

may be overridden by defining list specific to one or more modules.

2.5 Output description

At the end of the execution, the program will write the results of the simulation in the directory specified by `resultpath` , with the format specified by `output format`. The output variables are specified in a list `plot`, in each Module.

TiberCAD output is divided in two classes: **mesh-based** and **mesh-independent** quantities.

Mesh-based quantities are all the quantities associated with the nodes of the mesh, such as Fermi level, electron and hole density, conduction and valence band, etc., together with all the quantities associated with the elements of the mesh, such as current density.

The output values for these quantities are reported in the files `simname msh.ext`, where *simname* is the simulation module used for the calculations and `ext` is the extension of the chosen file format, e.g. `vtu` for paraview output.

`strain_msh.vtu`

In the case a sweep calculation is performed and the `plot data` keyword is set to true, the output files are of the kind `simname sweepvariable step value msh.ext`, where

sweepvariable is the variable with respect to which the sweep is performed (e.g. gate

voltage) and *step value* is the value of this variable at that step; e.g the result at the step *Vbias = 1.1* will be found in the file:

`dd_Vbias_1.1_msh.vtu`

mesh-independent quantities are the quantities which are not associated to the mesh, for example current at the contacts of a diode or quantized energy levels in a quantum well. These **mesh-independent** quantities are displayed in separated files, with the format `simname.ext`, e.g `quantum_electrons.dat` , where *simname* is the name of the model (simulation) associated to the results. If a sweep is performed, the output file gets the format `sweep name simname.ext`, where *sweep_name* is the name of the sweep performed, for example

`sweep_drain_driftdiffusion.dat`

Inside the file, output values for all the steps of calculation are shown.

2.6 Example of Input file

Here is an example of the input file template:

```
Device hemt1
{
  meshfile = hemt_msh.grd
  mesh_units = 1e-6
  material = GaN
  y-growth-direction = (0,0,0,1)
  z-growth-direction = (1,0,-1,0)
  x-growth-direction = (-1,2,-1,0)
  Region barrier
  {
    material = AlGaIn
    x = 0.14
  }
  Region barrier_doped
  {
    mesh_regions = (barrier_dop_s, barrier_dop_d)
    material = AlGaIn
    x = 0.14
    Doping
    {
      density = 1e21
      type = donor
      level = 0.026
    }
  }
  Region buffer { }

  Region buffer_doped
  {
    mesh_regions = (buffer_dop_s, buffer_dop_d)
    Doping
    {
      density = 1e21
      type = donor
      level = 0.026
    }
  }
  Region cap
  {
    Doping
    {
      density = 5e18
      type = donor
    }
  }
}
```

```

        level = 0.026
    }
}
Region cap_doped
{
    mesh_regions = (cap_dop_s, cap_dop_d)
    Doping
    {
        density = 1e21
        type = donor
        level = 0.026
    }
}
Region passivation
{
    mesh_regions = (passiv1, passiv2, passiv3, passiv4)
    material = SiN
}
Cluster semiconductor
{
    regions = -passivation
}
}
Module driftdiffusion
{
    name = dd
    regions = all
    coupling = electrons
    save_state = true
    #load_state = output_Nt7e16_out/dd_Vd_50.tsv
    statistics = FermiDirac
    strain_simulation = macrostrain
    plot = (Ec, Ev, Eg, eQFermi, hQFermi, eDensity, hDensity, Polarization,
           eCurrentDensity, hCurrentDensity, CurrentDensity, ContactCurrents,
           ElPotential, ElField, eMobility, hMobility, eConductivity, hConductivity)

    NonlinearSolver linesearch
    {
        # type = ls
        relative_tolerance = 1e-15
        step_tolerance = 5e-3
        max_iterations = 15
        LinearSolver petsc
        {
            ksp_type = pconly
            preconditioner = lu
        }
    }
}

```

```
    }
Physics
{
    recombination (srh, direct, auger) { }
    electron_mobility field_dependent
    {
        region = buffer
        low_field_model = constant
    }
    electron_mobility doping_dependent
    {
        region = (barrier, barrier_doped, cap, cap_doped, buffer_doped)
    }
    trap fixed_charge
    {
        region = GaN/SiN
        Nt = 2.74e13
    }
    trap acceptor
    {
        region = buffer
        Nt = 7e16
        Et = 0.5
        reference = cb
    }
    polarization (piezo, pyro) { }
}

Contact drain
{
    voltage = @Vd
}
Contact source
{
    voltage = 0.0
}
Contact gate
{
    regions = (gate1, gate2)
    type = schottky
    work_function = 1.5272
    voltage = @Vg
}
}

Module macrostrain
{
```



```

regions = -passivation # all the device except Region "passivation"
plot = all
LinearSolver # default
{
    tolerance = 1e-7
    max_iterations = 10000
    #xmonitor = true
    pc = composite
}
Physics
{
    Boundary substrate
    {
        regions = bottom
        material = GaN
        y-growth-direction = (0,0,0,1)
        z-growth-direction = (1,0,-1,0)
        x-growth-direction = (-1,2,-1,0)
    }

    Boundary extended_material
    {
        regions = side
    }
}

Module sweep
{
    name = sweep_g
    variable = Vg
    solve = sweep_d
    start = -1
    stop = 1.0
    steps = 2
    max_step = 0.25
    min_step = 1e-4
    #plot_data = true
}

Module sweep
{
    name = sweep_d
    variable = Vd
    solve = driftdiffusion
    start = 0.0
    stop = 50.0
}

```

```
steps = 200
min_step = 1e-4
plot_data = true
}
```

Module selfconsistent

```
{
  name = relaxation
  solve = (macrostrain, driftdiffusion)
  absolute_tolerance = 1e-3
  relative_tolerance = 1e-5
}
```

Simulation

```
{
  temperature = 300
  verbose = 3
  solve = (macrostrain, dd, sweep_g)
  logfile = output_Nt5e16_Al0.215_SiN/hemt.log
  # these parameters can be defined in modules, too
  resultpath = output_Nt5e16_Al0.215_SiN
  output_format = vtk
}
```

ELASTICITY

3.1 Abstract

Elasticity is a Finite Elements solver for mechanical equilibrium problems. It brings features typically developed for structural stability solvers into device modeling. The coupled treatment of the electro-mechanical problem within a unique framework results very useful to explore for multidisciplinary ideas.

3.2 Mini theoretical intro

Assuming a small displacement, Elasticity elasticity computed the strain by solving the equation

$$\frac{\partial}{\partial x_j} C_{ijkl} \frac{\partial u_l}{\partial x_k} = f_i$$

where C is the stiffness constant and f is the total external body force. The latter may include several contribution such as the converse piezoelectric effect the thermal expansion and a lattice mismatch induced strain. The latter, described in the example below, occurs when an interface is created between two materials with different lattice constants. Let ϵ^{LM} be the lattice mismatch strain, then the effective body force reads as :

$$f_i = -\frac{\partial}{\partial x_j} C_{ijkl} \epsilon_{lk}^{LM}$$

3.3 Example

In the following we will compute the strain induced by the lattice mismatch in a system comprising a layer of GaN between two contacts of $Al_xGa_{1-x}N$. Once the mesh is created with gmsh (dis-

tributed along TiberCAD) we may start to build the input file. First, we have to insert the region section

```
Device
{
  dimension = 3
  meshfile = mesh.msh
  mesh_units = 1e-9
  material = AlGaN
  x = 0.2
  x-growth-direction = (-1,0,1,0)
  y-growth-direction = (-1,2,-1,0)
  z-growth-direction = (0,0,0,1)
  Region Well {material = GaN}
}
```

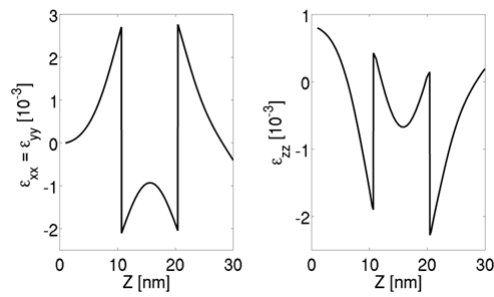
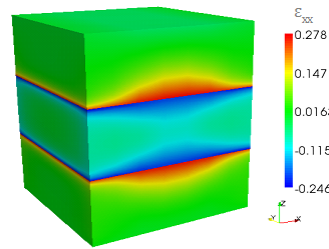
All the options included in this section, such as the material, axis growth and alloy concentration, apply for the whole structure. If we want to specify a region with different properties, we may use the keyword **Region** and refer to a region name, as specified in the mesh file.

The second part of the input file is devoted to the module declaration

```
Module elasticity
{
  Physics
  {
    BodyForceLattice_mismatch
    {
      x-growth-direction = (-1,0,1,0)
      y-growth-direction = (-1,2,-1,0)
      z-growth-direction = (0,0,0,1)
      reference_material = AlGaN
      x = 0.2
    }
    Contact Base {type = clamp}
  }
}
```

In physics we declare the physical models to be applied to our device. In our case, with **BodyForceLattice_mismatch** we are adding a body force into the system, induced by the lattice mismatch with the reference lattice which in this case is $Al_{0.2}Ga_{0.8}N$. In order to avoid a free standing device we may want to freeze a surface. With **Boundary Clamp** we fix, in this case, the region **Base**.

The third part is simply **Simulation {solve = elasticity}** where the default name *elasticity* is used. By default, files for 3D system are written in vtu which can be read from Paraview. Output data about strain are shown below.



DRIFT DIFFUSION

In this example we will see a very simple TiberCAD simulation:

1D calculation of Poisson and drift-diffusion for a bulk Silicon sample.

The following files should be in your working directory:

bulk.tib : **TiberCAD** input file

bulk.msh : mesh file

The mesh file can be obtained from the following GMSH geo file : **bulk.geo**

This is the summary of the Sections of this Tutorial :

- *Step 1 - Modeling the device*
- *Step 2 - Meshing the device*
- *Step 3 - TiberCAD Input file*
- *Step 4 - Run TiberCAD*
- *Output*

4.1 Step 1 - Modeling the device

As a first step, we have to model the device. To do so, you can use DEVISE module of ISE-TCAD 9.5 software package or GMSH program.

Here we'll see in details the procedure for GMSH.

There are two possible ways to use GMSH:

Interactive, using the graphical interface Using a script file In the following we'll see how to write a basic GMSH script (bulk.geo); for any details please refer to GMSH manual GMSH (<http://geuz.org/gmsh/>).



: In a **1D** simulation it is assumed that the geometrical model is restricted to the **x axis** . In a **2D** simulation it is assumed that the geometrical model is restricted to the **xy-plane (z=0)** Any other geometrical orientation could give unpredictable results

In a GMSH script, several variables can be defined and given a value in this way:

```
L = 1  
d = 0.01
```

these are valid GMSH variables: L is just the length of the Si sample; d is the value of a *characteristic mesh length* (see below).

Definition of geometrical entities **Points**

```
Point(1) = {0, 0, 0, d}  
Point(2) = {L, 0, 0, d}
```

The first three expressions inside the braces on the right hand side give the three X, Y and Z **coordinates** of the point; the last expression (**d**) sets the **characteristic mesh length** at that point, that is the “*size*” of a mesh element, defined as the length of the segment for a line segment, the radius of the circumscribed circle for a triangle and the radius of the circumscribed sphere for a tetrahedron.

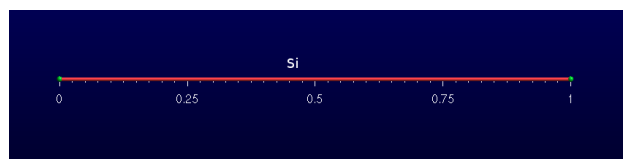
Thus, the smaller is the value of d, the greater is the mesh density close to that point.

The size of the mesh elements will then be computed in GMSH by linearly interpolating these characteristic lengths in the whole mesh.

Definition of a geometrical entity **Line**

```
Line(1) = {1, 2}
```

The two expressions inside the braces on the right hand side give the identification numbers of the start and end points of the line.



Definition of the physical entity **Physical Line bulk**

```
Physical Line("bulk") = {1}
```

The expression(s) inside the braces on the right hand side give the identification numbers of all the **geometrical lines** that need to be grouped inside the *physical line*.

In this way, in general, physical regions are created which associate together geometrical regions, and then the related mesh elements, which share some common physical properties. It's only these physical regions which can be referred to outside GMSH. In TiberCAD, this is done by associating one or more physical regions to a **TiberCAD region** through the keyword *mesh_regions* (see in the following).

Definition of two physical entities Physical Point:

```
Physical Point("anode2") = {1}
Physical Point("cathode") = {2}
```



: In general, in a nD simulation, **(n-1)D** physical regions (points in 1D, lines in 2D, surfaces in 3D) are used by TiberCAD to impose the required boundary conditions.

Each $(n-1)D$ physical region defined in this way in GMSH will be associated in TiberCAD to a boundary condition region, through the keyword **BC_reg_numb**. Thus, in this case, Physical points Anode and Cathode will be associated respectively to two **Contact** regions (see in the following).

4.2 Step 2 - Meshing the device

The *.geo* script file with the geometrical description can be run in GMSH, to display the modelled device and to mesh it through the GMSH graphical interface. Alternatively, a *non-interactive* mode is also available in GMSH, without graphical user interface.

For example, to mesh this 1D tutorial in non-interactive mode, just type:



```
gmsb bulk.geo -1 -o bulk.msh
```

where *bulk.geo* is the geometrical description of the device with GMSH syntax:

-1 means *1D mesh generation*

some command line options are:

- 1 , -2, -3 to perform *1D*, *2D* or *3D* mesh generation,
- o **mesh_file.msh** to specify the name of the mesh file to be generated

In this way, a .msh has been generated and is ready to be read in TiberCAD.

4.3 Step 3 - TiberCAD Input file

Now we have to write down the **TiberCAD input file** ([bulk.tib](#)). For a detailed reference see the user guide (Input file) and Getting started 1

4.3.1 Description of Device Regions

First, we have to list all the **TiberCAD Regions** present in our Device: a TiberCAD Region is usually a section of the device featuring the same material and possibly the same doping.

```
Device
{
  meshfile = bulk.msh

  Region bulk
  {
    material = Si

    Doping
    {
      Nd = 1e16
      type = donor
    }
  }
}
```

The TiberCAD Region bulk is made of Silicon and n-doped with a concentration $1 \times 10^{16} \text{cm}^{-3}$.

Through the keyword **Region**, one GMSH physical region (Physical Lines in 1D, Physical Surfaces in 2D, Physical Volumes in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Region, in this way:

```
Region GMSH_physical_region_name
```

In this case, the Physical Line bulk is associated to the TiberCAD Region bulk.

Alternatively, through the optional keyword **mesh_regions**, one or more GMSH physical regions can be associated to a single TiberCAD Region.

4.3.2 Definition of Simulation

Now we define the **Simulation** `driftdiffusion_1`: it belongs to the class **driftdiffusion**

```
Module driftdiffusion
{
  name = driftdiffusion # this is the default name

  regions = all          # 'all' is the default

  # what we want to plot
  plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
  ....
}
```

The TiberCAD simulation *driftdiffusion_1* , belonging to the model `driftdiffusion`, will be applied to the whole device structure (`regions = all`)

4.3.3 Definition of Boundary Conditions

The anode and cathode contacts of our 1D Si sample are defined as **Boundary conditions regions** (`Contact anode`, `Contact cathode`) in the following way:

```
Module driftdiffusion {
  name = driftdiffusion
  regions = all
  plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)
  Contact anode { voltage = $Vb } Contact cathode { }
}
```

Through the keyword `Contact` , one (n-1) -dimension GMSH physical region (Physical Point in 1D, Physical Line in 2D, Physical Surface in 3D) previously defined in the GMSH mesh ([Step 1 - Modeling the device](#)), can be associated to the present TiberCAD Contact, in this way:

```
Contact GMSH_physical_region_name
```

In this case, the *Physical Point* **anode** is associated to the TiberCAD Contact anode and the Physical Point cathode is associated to the TiberCAD Contact cathode.

Both contacts are defined as *ohmic* , cathode is assigned a fixed `voltage = 0.0` , while anode voltage is given by the value of the variable *Vb*

```
voltage = @Vb
```

4.3.4 Definition of Simulation parameters

Vb is specified in the sweep block, in the Solver section:

```
Module sweep
{
  solve = driftdiffusion
  variable = $Vb
  start = 0.0
  stop = 1
  steps = 10
  plot_data = true
}
```

In this way, the simulation `driftdiffusion_1` is performed for 10 (`steps = 10`) values of the anode voltage (variable = Vb), between 0 and 1.

For each step we want to plot the solution variables specified in the `driftdiffusion` module (`plot_data = true`).

4.3.5 Definition of Execution parameters

In the **Simulation** section , we decide *which* simulations to perform and in which *order*; set `solve = sweep` , to execute the sweep which run `driftdiffusion_1` simulation for the specified loop.

```
Simulation
{
  verbose = 2

  solve = sweep

  resultpath = output
  output_format = grace
}
```

Output files with conduction and valence band profiles (plot = Ec,Ev..) and all the calculated values of the current at the contacts (*ContactCurrents*) (the IV characteristic) are generated.

To increase the amount of information written to the screen we can vary the verbose level (verbose = 2).

4.4 Step 4 - Run TiberCAD

Now we can run TiberCAD

by double clicking on `bulk.tib` file (in Windows)

or by command line in linux: `tibercad bulk.tib`

4.5 Output

The generated Output files are:

- `driftdiffusion_materials.dat` : material (mesh) regions, in this case just region 1
- `driftdiffusion_nodal.dat` : nodal quantities (here conduction and valence band)
- `sweep_driftdiffusion_Vb.dat` : integrated current at the two contacts for each sweep step

Attachment Size

- `bulk.tib` 1.16 KB
- `bulk.geo` 181 bytes
- `bulk.msh` 4.49 KB

THERMAL

5.1 Introduction

The steady state heat flux, within the diffusive approach, is given by the Fourier's law, i. e. $J_i = -\kappa_{ij} \frac{\partial T}{\partial x_j}$ where κ is thermal conductivity second-rank tensor (which is assumed temperature independent).

The power balance is ensured by the continuity equation for the thermal flux $\frac{\partial J_i}{\partial x_i} = H$ where H is the total heat source.

Thermal module computes the balance between the heat generated and the heat dissipated. Here we get the temperature profile of a rectangular device with an hotspot in the center.

The device block specifies the material (Silicon) and the physical regions. In this case we have the HotSpot and the Base region

```
Device
{
  material = Si
  Region HotSpot{}
  Region Base{}
}
```

Let's now declare the thermal module. The temperature is fixed at the left and right side of the domain. The hotspot is included only in the HotSpot region

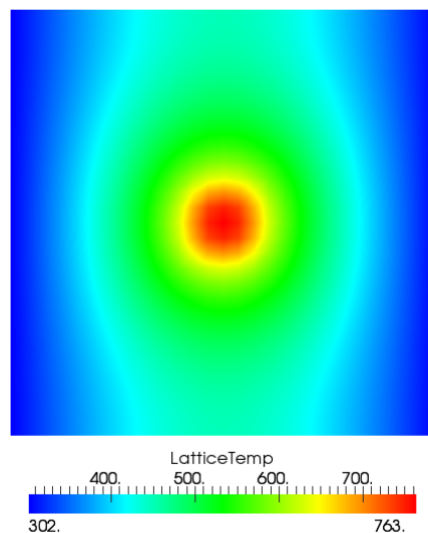
```
Module thermal
{
  Physics
  {
    Contact left
    {
      type = heat_reservoir
      temperature = 300
    }
  }
}
```

```
Contact right
{
  type = heat_reservoir
  temperature = 300
}
HeatSource constant
{
  regions = HotSpot
  H = 1e10
}
}
```

In the simulation region we write which modules should be solved.

```
Simulation{solve = thermal}
```

The temperature map is shown below



5.2 Boundary conditions

The boundaries of the thermal simulation domain are set to ideally thermal insulator by default.

Dirichlet boundary conditions can be imposed with the keyword `heat_reservoir`.

Example:

```
Contactheat_reservoir
{
  region = substrate
}
```



```
temperature = 300
}
```

The surface resistance can be added with the keyword `boundary surface_resistance`. (`r_surf` is in $\text{cm}^2 \text{ W/K}$).

Example:

```
Contactsubstrate
{
  type = surface_resistance
  r_surf = 0.05
  temperature = 300
}
```

5.3 Heat sources

The constant value heat source can be included with `heat_source constant`. The heat source must be in W/cm^3 .

Example:

```
HeatSourceconstant
{
  region = qdot
  H = 1e10
}
```

Heat generated by the Joule's effect is included with `heat_source joule`. The name of the drift-diffusion simulation is given by `dd_simulation`.

Example:

```
HeatSourcejoule
{
  dd_simulation = dd
}
```


QUANTUM EFA CALCULATIONS

In TiberCAD, it is possible to perform quantum calculations in the framework of Envelope Function Approximation (EFA): eigenstates and eigenfunctions of a given system, dispersion of quantum states and particle quantum density can be obtained respectively by means of the following Modules:

- Module `efaschroedinger`
- Module `quantumdispersion`
- Module `quantumdensity`

The optical properties are calculated by the following modules

- Module `opticskp`
- Module `opticalspectrum`

6.1 Module `efaschroedinger`

The EFA calculation of eigenstates and eigenfunctions are performed by the **Module** `efaschroedinger`. A typical example is the following:

```
Module efaschroedinger
{
  particle = el
  poisson_model_name = dd
  strain_model_name = strain # macrostrain
  name = quantum_el
  regions = quantum
  plot = (EigenFunctions, EigenEnergy, EnergyLevels)
  Solver
  {
    number_of_eigenstates = 10 # 30
  }
}
```

```
Physics
{
  model = conduction_band
}
}
```

6.1.1 Module options

The following options influence the behaviour of the Module `efaschroedinger`:

particle = string defines for which particle (electron or hole) Schroedinger equation is solved. Possible values are `el` and `hl`. A different Module `efaschroedinger` has to be defined for each particle to be solved.

poisson_model_name = string defines the name of the simulation (Module `driftdiffusion`) that can provide electric potential

strain_model_name = string defines the name of the simulation (Module `macrostrain`) that can provide elastic strain

regions = string defines the regions associated to this EFA simulation

6.1.2 Solver section

The Solver section of the Module `efaschroedinger` contains the following options:

number_of_eigenstates = integer defines the number of eigenvalues and eigenfunctions to be found.

Dirichlet_bc_everywhere = boolean : if true (default value), Dirichlet boundary

conditions are imposed over all the boundaries of the simulation region

solver = string : defines the solver for the eigenvalue problem, possible values are:

`arnoldi`, `lapack`, `krylovshur`. The default value is **krylovshur**. In the case of the `lapack` solver all the eigenvalues are computed. In the case of `arnoldi` or `krylovshur` solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, referred to as the *spectral_shift*

max_iteration_number = integer : maximum number of iteration, used as a stop condition

eigen_solver_tolerance = double : numerical eigensolver tolerance used as a convergence criteria

```
spectral_shift = double :the closest eigenvalues to this value (eV) are
found. If not
```

defined, then it will be calculated internally from the band edges.

```
ksp_type = string : Krylov subspace method type; bcgsl, gmres, cg
pc_type = string : preconditioner type: cholesky, jacobi, ilu , composite.
```

6.1.3 Physics section

```
model = string : possible values are : conduction band , for single conduction
band
```

model (Γ point) ; kp for $\mathbf{k} \cdot \mathbf{p}$ model

```
kp_model = string : possible values are : 6?6 , 8?8.
```

6.1.4 Output

The available output variables, to be specified in the plot option, are the following:

- EigenEnergy Eigen energy in eV
- EigenFunctions $|\psi(\mathbf{r})|^2$ function of the eigenstate
- Occupation probability to find the state occupied. It is calculated assuming Fermi

distribution and mean electrochemical potential and temperature:

- EnergyLevels graphical output used for showing the energy level over the band

diagram.

6.2 Module quantumdispersion

With the Module quantumdispersion it is possible to calculate the dependence of quantum eigenstates on $\mathbf{k}$ -vector. Such dependence gives the *quantum state dispersion* . The simulation name is **quantumdispersion** .

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
```

```
wedge = half
k_space_dimension = 1
k1 = (0, 0.1, 0)
number_of_nodes = (10)
output_format = grace
plot = k-space_dispersion
}
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

- **quantum simulation** : name of the efascroedinger simulation.
- **min eigenvalue number** , **max eigenvalue number** : the dispersion is calculated

for the states number i , where

$$\text{max_eigenvalue_number} \geq i \geq \text{min_eigenvalue_number}$$

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

6.2.1 Output

The output variable name is **k-space_dispersion** . The output format for the dispersion can be controlled independently of the general specification in the `Simulation` section by redefining the `output_format` keyword.

6.3 Module quantumdensity

In Module `quantumdensity` you can define the calculation of particle (electron,hole) density, based on the result of a previous calculation of the system eigenstates (e.g. with `efascroedinger` module).

```
Module quantumdensity
{
  name = dens_el
  regions = quantum
  plot = quantum_density
  k-space_dimension = 0
  k-space_basis = true
  k1 = (0, 0, 0.1)
  #number_of_nodes = (4)
  #wedge = half
  quantum_simulation = quantum_el
  degeneracy = 2
}
```

```

refine_fraction = 0.20
relative_accuracy = 0.01
refine_k_space = true
output_density_in_k_space = true
uniform_refinement = false
mesh_order = FIRST
first_state = 3
initial_eigenstates_number = 10
analytic = true
}

```

The available options are:

- **quantum_simulation** : name of the efaschroedinger simulation.
- degeneracy: degeneracy of the quantum state
- **initial_eigenstates_number** : initial number of eigenstates for the Schroedinger

equation

- **analytic** = { true | false } If true then the density is calculated analytically, otherwise numerically.

6.3.1 Output

The output parameter is **quantum_density**.

6.4 Module opticskp

By defining the Module **opticskp** , calculation of optical properties is enabled.

```

Module opticskp
{
  name = optics
  regions = quantum
  plot = (optical_spectrum_k_0 )
  initial_state_model = quantum_el
  final_state_model = quantum_hl
  initial_eigenstates = (0, 9)
  final_eigenstates = (0, 15)
  polarization = (0, 0, 1)
  Emin = 2.8
  Emax = 3.6
  dE = 0.001
}

```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

6.4.1 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for $k=0$.

6.5 Module opticalspectrum

By defining the Module **opticalspectrum**, optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

`k_space_dimension = 1` for 2D simulations, `2` for 1D simulations. `k-space basis` is

true if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in real space

If `refine_k_space = true` , that is adaptive k-mesh refinement is enabled, all the el-

ements whose error is greater than the value $(1-\text{refine fraction}) * (\text{maximum error})$ are going to be refined. In this case, Error is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

`number_of_nodes` = numb. of elements in k mesh, along each direction

`wedge = half | quarter`, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin, Emax, dE` : energy range and step of spectrum calculation.

6.5.1 Output

The output variables for optics calculations are:

- **optical_spectrum** : k-space integrated optical emission spectrum.

SIMULATION DYE SOLAR CELLS

The DSC model is tagged as *dssc* . In **options** subsection we indicate the simulation name:

```
model dssc
{
  options
  {
    simulation_name = <name_of_the_model>
    plot(Potential, Density, Current, ContactCurrents)
  }
  BC_regions
  {
    ...
  }
}
```

The boundary conditions are inserted in the **Model** section, the two contacts are the photoanode and the cathode. The photoanode must be the boundary of a region where TiO_2 is present, on the contrary the cathode must be the boundary of a region where the **electrolyte** is present.

```
BC_regions
{
  BC_region <name_of_the_anode>
  {
    type = ohmic
    bias = @V[0.0]
  }
  BC_region <name_of_the_cathode>
  {
    type = Pt
    load = @R[1e8]
  }
}
```

The anode is an ohmic contact, it contains only the sweep of the bias applied for the electrochemical potential of the electrons. The cathode must contain the initial external resistance (**load = R**).

If a simulation of the cell under illumination is made the initial value of *load* must be set to a high value, in the range of $10^8 \omega cm^2$ in order to have no current during the light intensity sweep. In case instead the simulation performed is under dark conditions where an external bias only is applied $load = 1.0$.

The generation term is related to the flux of photons which reaches the active TiO_2 regions and the dye present in the cell. We assume a simple Beer-Lambert exponential decay for charge generation of the form:

$$G(\mathbf{r}) = \int_{\lambda_1}^{\lambda_2} \alpha(\lambda) \Phi(\lambda) e^{-\alpha(\lambda) \hat{n} \cdot \mathbf{r}} d\lambda \quad (7.1)$$

where α is the absorption coefficient (in μ^{-1}) of the chosen Dye, $\Phi(\lambda)$ the intensity of the light power at that wavelength for the spectrum of the sun. The part of the input file for the generation model:

```
model dssc_generation
{
  options
  {
    regions = (<TiO2_region_name_1>, <TiO2_region_name_2>, ...)
    plot = (Distance, Generation)
    light_direction = <vector Indicating the direction of light>
    (example, illumination from x direction: light_direction = (1, 0, 0))
    light_intensity = @x
    dye = N719
  }
  BC_Regions
  {
    BC_Region <name_of_the_boundary>
    {
    }
  }
}
```

In the generation input file must be specified:

- (**regions**) the regions where we want that there is generation (where the Dye is present);
- (**plot**) if we want to plot the generation;
- (**light_direction**) the vector which fixes the direction from where the light comes;
- (**light_intensity**) the light intensity;
- (**light_intensity**) the light intensity;
- (**dye**) the dye used in the cell;

The light intensity is defined in a sweep (see section [Solver](#)). It can be set to reach 1 that means one Sun, or a larger or smaller illumination intensity (0.1, 2.0, etc.). There is another flag called

illumination_spectrum that can be used if we want to change the spectrum profile F with another illumination source (for example if we assume the cell under concentration of light). The file used by default for Φ is the standard 1.5 AM spectrum of the sun contained in the material database in the file `Sun1p5am`.

The last part of the generation is the definition of the boundary. This boundary says to the code which is the boundary region of the grid from where the light penetrates in the device. The information given by the boundary and the light direction vector defines the coming of light and direction of rays.

7.1 Device

Device section for a DSC simulation:

```
Device
{
  Region <name of the porous region>
  {
    mat = TiO2mes
    porosity = 0.5
  }
  Region <name of the electrolyte region>
  {
    mat = TiO2mes
    TiO2 = false
  }
}
```

For every region of the device it is specified the material file from the database (the `TiO2mes` contains standard parameters for both TiO_2 and electrolyte, so it can be used for both regions). For every region must be specified if it contains TiO_2 , or electrolyte or both.

This can be done setting two flags called `TiO2` and `electrolyte`. If set **true** the material is present in the region, if set **false** it is not present. By default they are assumed both **true** (porous region). In the second region of the example showed there is electrolyte only and `TiO2 = false` is explicitly specified. In case both materials are present (porous region) a porosity must be specified (in the range between 0, TiO_2 only, and 1, electrolyte only).

If one material is not present the porosity is automatically set to 0 or 1.

7.2 Physics

The set of parameters that can be defined for the entire device are enlisted in table 3.1. For the generation:

```
dssc
{
  generation = dssc_generation
}
```

7.3 Solver

Three sweeps are needed for the plot of the entire I-V under illumination. The first sweep is needed to make the transition from dark conditions to full open-circuit conditions under

Parameters		
	Poisson equation	
porosity	Porosity of porous material	
perm_oxide	Relative perm. of the TiO ₂	
perm_electrolyte	Relative perm. of the electrolyte	
k_3	Oxidized Dye regeneration rate constant	s ⁻¹
	Drift Diffusion equation	
ne	Dark electron density	cm ⁻³
nI	Dark iodide density	mol/l
nI3	Dark electron density	mol/l
mu_e	Electron mobility	cm ² /Vs
D_I	Iodide diffusion constant	cm ² /s
D_I3	Triiodide diffusion constant	cm ² /s
D_C	Cation diffusion constant	cm ² /s
	Recombination term	
k_e	Recombination constant rate	s ⁻¹
beta	Non-linearity exponent in the electron density recombination	

Table 7.1: Physical parameters that can be set for the model divided in subsets relative to different processes in the cell.

illumination. The high value of the load R maintains the current to zero. Then a second sweep is used to pass from open circuit condition to short circuit condition lowering the external load from a high external load to a small one ($R = 1$). Finally, the voltage sweep compute the I-V characteristics under illumination. In case of dark simulation (application of an external bias without illumination) the first and second sweep are not needed. The external load for the cathode can be set directly to $R = 1$.

```
Solver
{
  Sweep
  {
    sweep_gen
```

```
{
  simulation = (dssc_generation, dssc)
  variable = x
  values = (0, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
  plot_data = true
}
sweep_R
{
  simulation = dssc
  variable = R
  values = ( 1000, 100, 1)
  plot_data = true
}
sweep_V
{
  simulation = dssc
  variable = V
  values = (0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0)
  plot_data = true
}
}
```

The intensity of illumination can be changed sweeping the value of x different to 1 (1 = 1 Sun of power).

7.4 Output

The output that want to be plotted are enlisted in section Model within option. See *Table nodal* , *Table elemental* and *Table scalar* .

Nodal quantities		
	Potential	
eQFermi	Electron electrochemical potential	$eV (-e\phi_n)$
IQFermi	Iodide electrochemical potential	$eV (-e\phi_{I^-})$
I3QFermi	Triiodide electrochemical potential	$eV (-e\phi_{I_3^-})$
CQFermi	Cation electrochemical potential	$eV (-e\phi_C)$
ElPotential	Electrostatic potential	eV
Eredox	Electrolyte electrochemical potential	$eV (-e\phi_{Red})$
	Density	
eDensity	Electron density	cm^{-3}
IDensity	Iodide density	cm^{-3}
I3Density	Triiodide density	cm^{-3}
CDensity	Cation density	cm^{-3}
Generation	The net electron generation rate	$cm^{-3}s^{-1}$
NetRecombination	The net recombination rate	$cm^{-3}s^{-1}$

Table 7.2: Nodal quantities. The flags Potential and Density allow to plot all the electrochemical potentials and all the densities, including generation and recombination.

Elemental quantities		
	Current	
ElField	Electric Field	Vcm^{-1}
CurrentDensity	Total current density	Acm^{-2}
eCurrentDensity	Electron current density	Acm^{-2}
ICurrentDensity	Iodide current density	Acm^{-2}
I3CurrentDensity	Triiodide current density	Acm^{-2}
CCurrentDensity	Cation current density	Acm^{-2}

Table 7.3: Elemental quantities. The flag Current allows to plot all of them in the x, y and z components.

Scalar quantities		
ContactCurrents	Contact currents	$*^a$

^adepends on dimension and symmetry

Table 7.4: Scalar quantities.

Part II

Theory

ELASTICITY

Details about the theory of continuous elasticity applied to device mechanical deformation can be found in [\[Povolotskiy\]](#) .

TiberCAD computes the mechanical deformation of a body subjected to external forces by means of the equilibrium mechanical equation, i.e.

$$\frac{\partial \sigma_{ij}}{\partial x_j} = f_i$$

wheres σ is the stress second-rank tensor and f is the external body force applied to the system. As we will see below, any strain and stress source can be appropriately mapped into an equivalent body force.

The stress tensor σ is related to the mechanical strain by means of the constitutive relationships $\sigma_{ij} = C_{ijkl}\epsilon_{lk}$ where $\mathbf{C}$ is the stiffness fourth-rank tensor. The strain is related to the displacement u by the expression

$$\epsilon_{kl} = \frac{1}{2} \left(\frac{\partial u_l}{\partial x_k} + \frac{\partial u_k}{\partial x_l} \right)$$

Relying on the symmetry of the strain and stiffness tensor the final equation reads as

$$\frac{\partial}{\partial x_j} C_{ijkl} \frac{\partial u_l}{\partial x_k} = f_i$$

The non-linear strain is computed by applying the mechanical equilibrium equation on the deformed mesh. The number of the shape iteration is set by the keyword **shape_iteration** (default = 0, i.e. no deformation is performed) whereas the maximum tolerated error can be indicated with **shape_error** (default = $1e^{-3}$).

8.1 Stiffness constant

By default the stiffness constant is anisotropic. For the zincblende crystal structure the independent coefficients are (with the Voigt notation) C11, C12, C44.

$$C = \begin{pmatrix} C_{11} & C_{12} & C_{12} & 0 & 0 & 0 \\ C_{12} & C_{11} & C_{12} & 0 & 0 & 0 \\ C_{12} & C_{12} & C_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & C_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & C_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & c_{44} \end{pmatrix} \quad (8.1)$$

For the wurtzite structure we have C11, C12, C13 and C44.

$$C = \begin{pmatrix} C_{11} & C_{12} & C_{13} & 0 & 0 & 0 \\ C_{12} & C_{11} & C_{12} & 0 & 0 & 0 \\ C_{13} & C_{12} & C_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & C_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & C_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{C_{11}-C_{22}}{2} \end{pmatrix} \quad (8.2)$$

An isotropic stiffness can be included with the keyword **Stiffness isotropic** . In this case, the only independent parameters are the Young modulus (Youngin GPa) and the Poisson's ratio (Poisson).

$$C = \frac{E}{(1+\nu)(1-2\nu)} \begin{pmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{pmatrix} \quad (8.3)$$

While the anisotropic model is included by default, the isotropic one must be explicitly indicated

Example:

```
Stiffness isotropic
{
  Young = 129
  Poisson = 0.349
}
```

Boundary conditions:

Surfaces forces f^0 are applied by imposing $\sigma_{ij}n_j = f_i^0$ along the surface with normal n. This boundary condition can be used with the keyword **surface_force** . The force can be specified in GPa with force. An example is shown below

```
Contact base
{
  type = surface_force
  force = (0,0,0.5)
}
```

On the other hand, one may want to fix some surface of the device. The keyword **clamp** freezes all nodes of a given surface.

Example:

```
Contact substrate
{
  type = clamp
}
```

Body forces

A constant value body force can be included by means of the keyword **Bodyforceconstant** .

Example:

```
BodyForceconstant
{
  force = (0,1,0)
}
```

When two crystals with different crystal structure are put in contact the lattice mismatch between them may induce a strain ϵ^{LM} and, therefore, a stress $\sigma = C\epsilon^{LM}$.

This stress contribution acts as a body force

$$f_i = -\frac{\partial}{\partial x_j} C_{ijkl} \epsilon_{lk}^{LM}$$

The strain source can be computed only once the reference lattice is identified. The reference material and its growth axis can be included following the same syntax of the device section.

Example:

```
BodyForcelattice_mismatch
{
  reference_material = AlGaN
  structure = wz
  x = 0.2
  x-growth-direction = (1,0,0,0)
  y-growth-direction = (0,1,-1,0)
  z-growth-direction = (0,0,0,1)
}
```

In presence of an electric field there might develop an additional stress source due to the so-called converse piezoelectric effect, given by :

$$\sigma_{il}^{SC} = -e_{ijk}E_k$$

The converse piezoelectric effect can be included with the keyword **BodyForce_piezo** and an electrostatic simulation must be indicated.

The effective body force reads as :

$$f_i = -\frac{\partial}{\partial x_j}\sigma_{ij}^{CP}$$

Example:

```
BodyForceconverse_piezo
{
  poisson_simulation = dd
}
```

Considering all the above mentioned force sources, the plotted total stress and strain are :

$$\sigma_{ij} = C_{ijkl} \left(\frac{\partial u_l}{\partial x_k} + \epsilon_{lk}^{LM} \right) - e_{ijk}E_k$$
$$\epsilon_{kl} = \frac{1}{2} \left(\frac{\partial u_l}{\partial x_k} - \frac{\partial u_k}{\partial x_l} \right) + \epsilon_{lk}^{LM}$$

DRIFT-DIFFUSION SIMULATION OF ELECTRONS AND HOLES

9.1 Theory

The semi-classical transport simulation of electrons and holes is based on the drift- diffusion approximation (see [\[Selberherr\]](#)).

Beside the electric potential the electro-chemical potentials are used as variables such that the system of PDEs to be solved reads as follows

$$\begin{aligned} -\nabla(\varepsilon\nabla\varphi - \mathbf{P}) &= -e(n - p - N_d^+ + N_a^-) \\ -\nabla(\mu_n n(\nabla\phi_n + P_n\nabla T)) &= R \\ -\nabla(\mu_p p(\nabla\phi_p + P_p\nabla T)) &= -R \end{aligned} \tag{9.1}$$

P is the electric polarization due to e.g. piezoelectric effects and R is the net recombination rate, i.e. recombination rate minus generation rate. P_n and P_p are the electron and hole thermoelectric power, respectively. The models for the mobilities and the net recombination rates can be specified in the **physical_model** sections as described in the following.

9.2 Solution/Plot variables

The solution variables available for plotting and for interaction with other models are given in *Table Plotting variables* .

9.3 Module options

The following options influence the behaviour of the Drift-Diffusion module:

coupling = string defines which equations to solve. The default is full, meaning that the full system consisting of the Poisson, electron continuity and hole continuity equations is solved. Other possible values are poisson (for equilibrium calculations), electrons or holes. For the last two cases local equilibrium is assumed such that $\phi_n = \phi_p$.

enforce_local_charge_neutrality = bool If set to true, local charge neutrality will be enforced by setting the charge density to zero. This may be useful for solving the Poisson equation involving only dielectrics.

guess_el_qfermi = double If this option is set, then before resolving the system the given number will be set as a guess for the electron electro-chemical potential.

guess_hl_qfermi = double If this option is set, then before resolving the system the given number will be set as a guess for the hole electro-chemical potential.

default_boundary_condition = string With this option the user can control the default boundary condition for the electric field on all external boundaries without explicit boundary model. Possible values are zero field (default), or zero displacement. The two differ only in presence of electric polarization fields.

quadrature_rule = string This option allows to chose between trapezoidal and Gauss type numeric integration rules. The default rule is gauss, but in some cases trapez may prevent density peaks near badly resolved material interfaces.

save_state = boolean If set to *true* the current solution will be written to a compressed file after each solve. The file name follows the same rules as the result files, having file extension *.tsv*.

load_state = file Reload a formerly saved solution. filename can be a filename (to reside in the current working directory), a relative or an absolute path to a file. The file needs to have been created with *save_state = true*.

solve_after_load = boolean If set to true, the system will be solved after having reloaded a saved state. Otherwise it will not be solved, which is the default behaviour.



Currently the reload of saved solutions *only works correctly when using the identical mesh*. Otherwise there will be undefined behaviour or failure. Future releases will relax this restriction.

9.4 Solver section

The **Solver** section of the Drift-Diffusion module refers to a nonlinear solver.

9.5 Physics section

The Physics block contains generic options for the bulk physical model and the definition of submodels. The generic options are:

model = string Specify the semiconductor model to be used. Possible values are default and simple. The former uses k.p to calculate the (strain corrected) band parameters.

thermal_simulation = string If you are doing coupled electrothermal simulations, you have to specify the name of the thermal simulation providing the lattice temperature.

strain_simulation = string If you are doing simulations on strained systems, you can specify the name of a strain simulation. The band parameters will then be calculated taking local strain corrections into account.

relax_polarization = double With this option one can specify a relaxation factor for the electric polarization field. This can be useful if the amount of total electric polarization has to be treated as fitting parameter.

For the simple semiconductor model one has to provide conduction and valence band edges and the effective density of states masses (or the effective density of states itself) in the `Region` sections. The corresponding keywords are given in table 2.2 . In the following we describe the submodels. All submodels can be restricted to a subset of simulation regions.

9.5.1 Recombination/generation models

This section describes the currently available generation/recombination models. Note that all recombination models can be applied also to surfaces/interfaces. Recombination models are controlled by means of recombination submodel blocks inside the Physics section. Different recombination models having the same (or no) options can be enabled in a single statement writing:

```
recombination (model1, model2, ...) {}
```

Shockley-Read-Hall (SRH) recombination

The SRH recombination model can be enabled by defining a recombination submodel of type `srh`.

SRH recombination is defined as follows:

$$R_{SRH} = \frac{np - n_i^2}{(n + n_i e^{E^*/k_B T})\tau_p + (p + n_i e^{-E^*/k_B T})\tau_n} \quad (9.2)$$

$E^* = E_{trap} - (E_c + E_v)/2$ is the trap level with respect to the midband energy.

n_i is the intrinsic carrier density, τ_n and τ_p are the recombination times.

The parameters are taken from the material database. The recombination times are dependent on temperature and doping density, e.g.

$$\tau_n = \tau_n^0 \left(\frac{T}{T_0} \right)^{\alpha_n} e^{\beta(T/T_0 - 1)} \quad (9.3)$$

$$\tau_n^0 = \tau_{min,n} + \frac{\tau_{max,n} - \tau_{min,n}}{1 + (N/N_{ref})^\gamma} \quad (9.4)$$

where T_0 is the reference temperature (300 K). Table 2.3 shows the corresponding parameters for the material data files. The parameters for holes and electrons have to be specified in an array, e.g. $\tau_{min} = (1e-5, 3e-6)$

The recombination times and trap level can be overridden from the input file by using the keywords of Table 2.4.

The SRH recombination model can be applied also to surfaces and interfaces. In this case, you can provide the recombination velocities using the keywords `rec_velocity_n` and `rec_velocity_p` instead of `tau_n` and `tau_p`.

Direct (radiative) recombination

The direct recombination model can be enabled in the input file by defining a `recombination` submodel of type `direct`.

Direct recombination is modeled as follows:

$$R_{direct} = C(np - n_i^2) \quad (9.5)$$

The material data file and the input file use the same keyword `C` for the parameter `C`. The database value can be overridden from the input file as described for SRH recombination.

Auger recombination

The Auger recombination model can be enabled in the input file by defining a recombination submodel of type `auger`.

Auger recombination is modeled by the following equation

$$R_{auger} = (C_n n + C_p p)(np - n_i^2) \quad (9.6)$$

with temperature dependent parameters

$$C_{\{n,p\}} = \left(A + B \frac{T}{T_0} + C \left(\frac{T}{T_0} \right)^2 \right) (1 + H e^{-\{n,p\}/N_0})$$

The parameters `A`; `B`; `C`; `H` and N_0 are taken exclusively from the database. They are different for C_n and C_p and have to be specified as arrays with keywords `A`, `B`, `C`, `H`, `N0`, e.g. `A = (1e-31, 1e-32)`. The calculated values for C_n and C_p can be overridden from the input file by specifying values for the keywords C_n and C_p .

Optical generation

A very simple model for photoelectric generation of electron-hole pairs is implemented in TIBER-CAD. It is enabled by specifying a `generation` submodel of type `optical`. The model imposes a constant generation rate which has to be provided by the keyword `G` in units of $(cm * s)^{-1}$.



Note that the simulation usually should define a sweep on the value of `G` from 0 to the desired generation.

9.5.2 Thermoelectric power models

The thermoelectric power models are the same for electrons and holes. The keyword is `thermoelectric_power`, i.e.

```
thermoelectric_power [type]
{
}
```

The model keyword can be `constant` (i.e. the thermoelectric powers are read from the database) or `diffusivity_model` where the thermoelectric powers are computed by

$$P_n = -\frac{k_b}{q} \left(\frac{5}{2} + \frac{e\phi_n + E_c - e\varphi}{k_b T} \right) \quad (9.7)$$

$$P_p = \frac{k_b}{q} \left(\frac{5}{2} - \frac{e\phi_p + E_v - e\varphi}{k_b T} \right) \quad (9.8)$$

The default is $P_n = P_p = 0$

9.5.3 Mobility models

The models to be used for electrons and holes can be defined in a single submodel block or independently using two blocks. The corresponding keywords are `mobility` or

`electron_mobility` and `hole_mobility`, i.e.

```
mobility [type]
{
}
```

```
electron_mobility [type]
{
}
```

```

    }

hole_mobility [type]
{
}

```

When using the first approach, both carriers will use the same model, and parameters provided in the input file will also be used by both carriers. When mixing the different definitions, the blocks `electron_mobility` and `hole_mobility` will override the common `mobility` block.

The default model is the constant mobility model. The parameters for the different mobility models are needed for both electrons and holes. In the material files they are specified with a common keyword in arrays, e.g.

mobility	constants
	electrons holes
mu_max	(1400.0 , 250.0)
exponent	(1.0 , 2.1)

9.6 Constant mobility model

The constant mobility model (identifier `constant`) assumes a mobility which depends only on temperature by means of the following formula:

$$\mu_{const} = \mu_0 (T/T_0)^{-\gamma} \quad (9.9)$$

In the material data file μ_0 and γ have to be specified with the keywords `mu_max` and `exponent`. μ_0 can be overridden from the `physical_model` section using the keyword `mu` or from the `Region` sections using the keywords `mu_e` and `mu_h`.

9.7 Doping dependent mobility model

The doping dependent mobility model (identifier `doping_dependent`) implements two models for mobility depending on the total doping density and the temperature. The model that is used depends on the value of the `mobility_formula` parameter.

Model by Masetti et al. [4]

The model by Masetti et al. is identified by `mobility_formula = 1`. It uses the following formula:

$$\mu = \mu_{min,1} * e^{-P_c/N} + \frac{\mu_{const} - \mu_{min,2}}{1 + (N/C_r)^\alpha} - \frac{\mu_1}{1 + (C_s/N)^\beta} \quad (9.10)$$

where N is the total doping density and μ_{const} the mobility obtained from the constant mobility model. The parameters are specified in the material file as given in Table 2.5.

Model by Arora [5] The model by Arora is identified by `mobility_formula = 2`. It reads:

$$\mu = \mu_{min} + \frac{\mu_d}{1 + (N/N_0)^{A^*}} \quad (9.11)$$

with

$$\begin{aligned} \mu_{min} &= A_{min}(T/T_0)^{\alpha_m}, & \mu_d &= A_d(T/T_0)^{\alpha_d} \\ N_0 &= A_N(T/T_0)^{\alpha_N}, & A^* &= A_a(T/T_0)^{\alpha_a} \end{aligned}$$

The parameters are given in table below.

Field dependent mobility model

The field dependent mobility model describes the degradation of mobility at high driving fields. It is identified by the identifier `field_dependent`. The electric field component in direction of the current ow or the gradient of the electro-chemical potential can be chosen as driving force:

`driving_force = efield | grad_fermi | field_parameter`

The default driving force is the gradient of the corresponding electro-chemical potential $\nabla\phi$. `field_parameter` uses a field parameter given by $\sqrt{E \cdot \nabla\phi}$ as driving force.

The model is based on the Caughey-Thomas model, refined by Canali [6]:

$$\mu = \frac{\mu_{lowfield}}{\left(1 + \left(\frac{\mu_{lowfield}|\mathbf{E}|}{v_{sat}}\right)^\beta\right)^{1/\beta}} \quad (9.12)$$

with

$$\beta = \beta_0(T/T_0)^b$$

$|E|$ is the modulus of the driving field, $\mu_{lowfield}$ is the low-field mobility. For the latter one can specify the model to be used using the parameter `lowfield_model`. As default the doping dependent model is used. There are two models for `vsat`, identified with `Vsat_Formula = 1` and 2.

Formula 1 reads

$$v_{sat} = v_{sat,0}(T/T_0)^{-\gamma}$$

Formula 2 reads

$$v_{sat} = \max(A_{vsat} - B_{vsat}(T/T_0), v_{min})$$

The parameters for the field dependent mobility model are summarized in Table 2.7.

9.7.1 Polarization models

For simulations involving materials with nonzero electric polarization (such as nitrides) it is important to include the effect of polarization. This is done by specifying the models for spontaneous (pyro-) and piezoelectric polarization using the keywords `polarization` with the types `pyro` and `piezo`

```
polarization pyro {}
polarization piezo {}
```

As for all models, if they do not have individual options, they can be specified together by writing **polarization (pyro, piezo) {}**.

Spontaneous (pyro-) polarization

The spontaneous polarization model imposes a constant electric polarization $\mathbf{P}$ along the symmetry-breaking direction of the crystal. Crystals with wurtzite structure like Ni- trides have strong piezoelectric fields along the c-direction. The value of the polarization usually is taken from the database, but it can be overridden from the input file by specifying the option `Pz`, meaning the value of the spontaneous polarization along c-direction ([0001]). Alternatively, one can specify explicitly a polarization vector using the option

$\mathbf{P} = (\mathbf{Px}, \mathbf{Py}, \mathbf{Pz})$. This is useful to impose an arbitrary constant polarization field.

Piezopolarization

The piezoelectric polarization is strain induced and given by

$$P^{pz} = e_{ikl}\varepsilon_{kl} \quad (9.13)$$

where ε_{kl} is the strain tensor. The piezoelectric moduli e_{ikl} are stored in the database. The strain is obtained from the simulation specified in the `Physics` section, but it can be overridden by providing a name for the strain simulation inside the polarization block using the `strain_simulation` option.

9.7.2 particle density

Details for the calculation of the electron and hole densities can be given in the `particle_density` submodel. Its options are:

`particle = string` The particle this model is describing. Can be `electron` or `hole`.

`statistics = string` The statistics. Can be `fermidirac` (default) or `boltzmann`.

`quantum_density = string` The name of a quantum density simulation. This will use the quantum mechanical particle density in the regions it was calculated.

If a quantum density is used, then it is useful to define also an embracing region where the model gradually switches from a fully classical to a fully quantum density. The options for the embracing are specified in a block with keyword `embracing`. It accepts the following options:

`embracing_length = double` When the domain of the quantum simulation is smaller than the domain of the full simulation, the boundary conditions for the Schrodinger equation will disturb the transfer from classical to quantum density. By defining an embracing region of a certain extension (specified in meters), a gradual transition from classical to quantum density will be done instead of an abrupt one, using as effective density $\rho = x \cdot \rho_{\text{quantum}} + (1 - x) \cdot \rho_{\text{classical}}$. The default is no embracing region at all (zero extension).

`cutoff = double` if an embracing region is used, a part of this region near the boundary of the quantum region can be cut off so that only the classical density is considered in that part. `cutoff` is specified as a percentage of the embracing length and should therefore be between 0.0 and 1.0.

`plot_embracing_region = bool` Whereas the automatic creation of the embracing region in 1D is a very simple task, it is a more difficult one in higher dimensions. By setting this flag to true, the embracing region and the mixing coefficient x will be plotted for a visual control of the quality of the embracing region. The default is `false`.

9.7.3 Trap models

Currently single level traps are implemented in TiberCAD. Traps can be normally neutral or normally charged electron or hole traps, or a fixed charge. Common options for all models are

`type = string` (If not provided as second keyword). The type of traps. One of `eNeutral`,

`hNeutral`, `donor`, `acceptor` or `fixed_charge`.

`Nt = double` The trap density in cm^3 (or cm^2 for surface traps).

`Et` = double The trap level with respect to a reference energy.

`reference` = string The reference energy. The default is `m`. The trap energy in this case is given as $E_{trap} = E_{midgap} + Et$. Other possible values are $E_{trap} = E_c - Et$ or $E_{trap} = E_v + Et$.

`eNeutral` The trapped electron density is given by

$$n_t = \frac{N_t}{1 + \exp\left(\frac{E_{trap} - E_{F,n}}{k_B T}\right)}$$

`hNeutral` The trapped hole density is given by

$$p_t = \frac{N_t}{1 + \exp\left(-\frac{E_{trap} - E_{F,p}}{k_B T}\right)}$$

`donor` The density of ionized traps is given by

$$N_t^+ = N_t - \frac{N_t}{1 + \exp\left(\frac{E_{trap} - E_{F,n}}{k_B T}\right)}$$

`acceptor` The density of ionized traps is given by

$$N_t^- = N_t - \frac{N_t}{1 + \exp\left(-\frac{E_{trap} - E_{F,p}}{k_B T}\right)} \quad (9.14)$$

If traps are specified, the total charge density in the Poisson equation is modified to include the charged trap densities:

$$\rho = e \left(p - n + N_D^+ - N_A^- - \sum n_t + \sum p_t + \sum N_t^+ - \sum N_t^- \right) \quad (9.15)$$

Additionally, each trap induces a SRH recombination term of the form

$$R_t = N_t \frac{v_{th}^n \sigma^n v_{th}^p \sigma^p (np - n_i^2)}{v_{th}^n \sigma^n (n + n_1) + v_{th}^p \sigma^p (p + p_1)} \quad (9.16)$$

where $\sigma^{n,p}$ are the capture cross sections, $v_{th}^{n,p}$ the thermal velocities and (for Boltzmann statistics)

$$n_1 = n_{i,eff} \exp(E_{trap}/k_B T), \quad p_1 = n_{i,eff} \exp(-E_{trap}/k_B T) \quad (9.17)$$

9.7.4 Boundary conditions

Boundary conditions are implemented for ohmic contacts, Schottky contacts, free surfaces and interfaces. Contacts are boundary models that allow a nonzero normal electrical current. The applied voltage is specified with the option `voltage`. A variable can be assigned to this, using the `$`-syntax. On ohmic or schottky contacts one can define surface recombination velocities for

electrons and holes using the options `rec_velocity_e` and `rec_velocity_h`. This will impose Robin type boundary conditions for the continuity equations of the form

$$-\nabla(\mu_n n \nabla \phi_n) = v_n(n - n_0) \quad (9.18)$$

$$-\nabla(\mu_p p (\nabla \phi_p + P_p \nabla T)) = -v_p(p - p_0) \quad (9.19)$$

The options `zero_field`, `zero_grad_fermi_e` and `zero_grad_fermi_h` can be used, which when set to `true` will impose zero normal electric field and zero normal gradient of the electron and hole electro-chemical potential, respectively. The latter are special cases of surface recombination velocities ($v_{rec} = 0$).

Contacts are defined by blocks with keyword `Contact`, for example:

```
Contact anode
{
  type = ohmic
  [regions = (anode1, anode2)]
  voltage = $Vd
}
```

An area factor can be specified for contacts using the keyword `area_factor`. The contact current will be multiplied by this factor. For interfaces and surfaces, the same syntax can be used (optionally one can use the keywords `Interface` or `Boundary`), however they do usually not need to be defined explicitly.

Ohmic contact

The ohmic contact (identifier `ohmic`) has no further parameters.

Schottky contact

A Schottky contact (identifier `schottky`) has the additional parameter `barrier`, which signifies the energy difference between the semiconductor band edge and the fermi energy in the metal. As default, the barrier is taken with respect to the conduction band. By specifying `band = v` the barrier can be imposed with respect to the valence band (p- type contact). Alternatively, the metal work function can be defined using the keyword `work_function`. Note, however, that its value has to be aligned with the band energies given in the material files for the other materials. The `fixed_barrier` controls the behaviour of the barrier height for strained materials. If it is set to `true`, the barrier will be independent of strain (default behaviour). If it is set to `false`, the given barrier is used as barrier for the unstrained case and will depend on strain during simulation. If the metal work function is specified, the barrier will be strain dependent as default.



if the contact is touching different materials, one should specify the work function instead of the barrier.

Thermionic emission is by default switched on, but can be disabled by specifying `thermionic_emission = false`.

Interface/surface model

The free surface or interface model (identifier interface) can include surface charges due to traps and surface recombination. Their definition can be found in section (vedi sopra).

Each trap model will induce automatically a SRH recombination model as in the bulk case.

9.8 Schroedinger/Poisson/Drift-Diffusion calculations

TIBERCAD is able to do selfconsistent Schroedinger-Poisson or Schroedinger-Drift-Diffusion calculations. For this purpose, quantum_density has to be specified for at least one of the carriers, and a selfconsistent simulation should be defined in the Selfconsistent block. The following options { to be specified in the Physics section { control the behaviour of the selfconsistent simulation.

use_density_predictor = bool When set to true, a predictor-corrector scheme will be adopted in the selfconsistent cycle. The Poisson/Drift-Diffusion solver does not just take the particle densities as given by the Schroedinger calculation, but it will assume a dependency of the density on the potentials of the form

$$\rho(\varphi, \phi_n, \phi_p) = \frac{\rho_{\text{quantum}}(\varphi^0, \phi_n^0, \phi_p^0)}{\rho_{\text{classical}}(\varphi^0, \phi_n^0, \phi_p^0)} \rho_{\text{classical}}(\varphi, \phi_n, \phi_p) \quad (9.20)$$

where $(\varphi^0, \phi_n^0, \phi_p^0)$ are the potentials for which the quantum density was calculated. use_density_predictor = true is the preferred method for selfconsistent Schroedinger-Poisson/Drift-Diffusion calculations and is enabled by default.

9.9 Example

The following example shows the Drift-Diffusion module definition for a pn junction.

```
Module driftdiffusion
{
  name = dd
  #regions = (pside, nside)
  Solver linesearch
  {
  }
  Physics
  {
    recombination srh {}

    mobility doping_dependent {}
  }
}
```

```
Contact anode
{
    voltage = $Vd
}

Contact cathode
{
    voltage = 0
}
}
```

Listing 3: Models section for drift-diffusion

Key	Description	Units
Ec	Conduction band edge	eV
Ev	Valence band edge	eV
eQFermi	Electro-chemical potential of electrons	$\text{eV} (-e\phi_n)$
hQFermi	Electro-chemical potential of holes	$\text{eV} (-e\phi_p)$
Ec0	Conduction band edge without electric potential	eV
Ev0	Valence band edge without electric potential	eV
Eg	Band gap	eV
ConductionBands	Minima of all conduction bands	eV
ValenceBands	Maxima of all valence bands	eV
ElPotential	Electric potential	V
eDensity	Electron density	cm^{-3}
hDensity	Hole density	cm^{-3}
eMobility	Electron mobility	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
hMobility	Hole mobility	$\text{cm}^2\text{V}^{-1}\text{s}^{-1}$
eConductivity	Electron conductivity	S/cm
hConductivity	Hole conductivity	S/cm
ElField	Electric Field	Vcm^{-1}
Polarization	Electric Polarization	Cm^{-2}
CurrentDensity	Total current density	Acm^{-2}
eCurrentDensity	Electron current density	Acm^{-2}
hCurrentDensity	Hole current density	Acm^{-2}
IonizedDonors	Ionized donor density	cm^{-3}
IonizedAcceptors	Ionized acceptor density	cm^{-3}
IonizedElectronTraps	Ionized electron trap density	cm^{-3}
IonizedHoleTraps	Ionized hole trap density	cm^{-3}
eThElPower	Electron thermoelectric power	V K^{-1}
hThElPower	Hole thermoelectric power	V K^{-1}
NetRecombination	The net recombination rate for each recombination/generation model and the total rate	$\text{cm}^{-3}\text{s}^{-1}$
ContactCurrent	The electric current on each contact	A/cm^{3-d}

Table 9.1: Solution/Plot variables

<i>keyword</i>	<i>description</i>
Ec	conduction band edge (eV)
Ev	valence band edge (eV)
m_dos_e	conduction band effective DOS mass (m_e)
m_dos_h	valence band effective DOS mass (m_e)
Nc	conduction band effective DOS (cm^{-3})
Nv	valence band effective DOS (cm^{-3})

Table 9.2: Parameters for the simple semiconductor model

<i>parameter</i>	<i>keyword</i>
τ_{min}	taumin
τ_{max}	taumax
N_{ref}	Nref
γ	gamma
E^*	Etrap
α	Talpha
β	Tcoeff

Table 9.3: SRH material data file parameters

τ_{n}	tau_n
τ_{p}	tau_p
E^*	E_t

Table 9.4: SRH input file parameters

<i>parameter</i>	<i>keyword</i>
$\mu_{min,1}$	mumin1
$\mu_{min,2}$	mumin2
μ_1	mul
P_c	Pc
C_r	Cr
C_s	Cs
α	alpha
β	beta

Table 9.5: Data file parameters for the mobility model by Masetti et al.

<i>parameter</i>	<i>keyword</i>
A_{min}	mumin
A_d	mud
A_N	N0
A_a	A
α_m	am
α_d	ad
α_N	aN
α_a	aA

Table 9.6: Data file parameters for the mobility model by Arora.

<i>parameter</i>	<i>keyword</i>
β_0	beta0
b	betaexp
$v_{sat,0}$	vsat0
γ	vsatexp
A_{vsat}	A_vsat
B_{vsat}	B_vsat
v_{min}	vsat_min

Table 9.7: Data file parameters for the mobility model by Arora.

THERMAL

Details about irreversible thermodynamics can be found in [\[Wachutka\]](#) .

The heat flux, within the diffusive approach, is given by the Fourier's law, i.e.

$$J_i = -\kappa_{ij} \frac{\partial}{\partial x_j} T \quad (10.1)$$

where κ is thermal conductivity second-rank tensor (which is assumed temperature in dependent). The power balance is ensured by the continuity equation for the thermal

$$\frac{\partial}{\partial x_i} J_i = H \quad (10.2)$$



10.1 Boundary conditions

The boundaries of the thermal simulation domain are set to ideally thermal insulator by default. Dirichlet boundary conditions can be imposed with the keyword *heat_reservoir* .

```
Contact reservoir
{
  temperature = 300
}
```

A surface boundary resistance can be added with the keyword *surface_resistance* .

```
Contact surface_resistance
{
  r_surf = 0.01
  temperature = 300
}
```

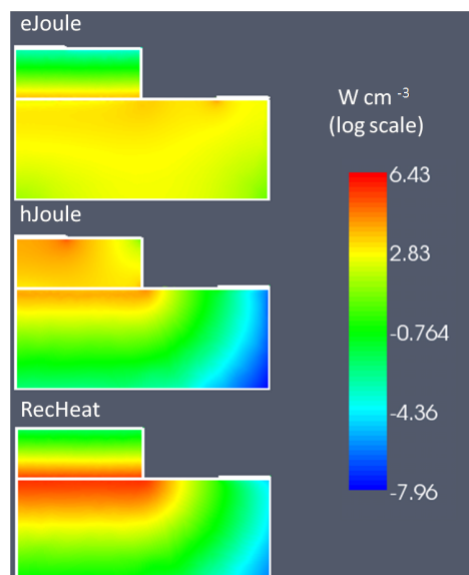
10.2 Heat sources

The constant value heat source can be included with *heat_source constant* .

Example:

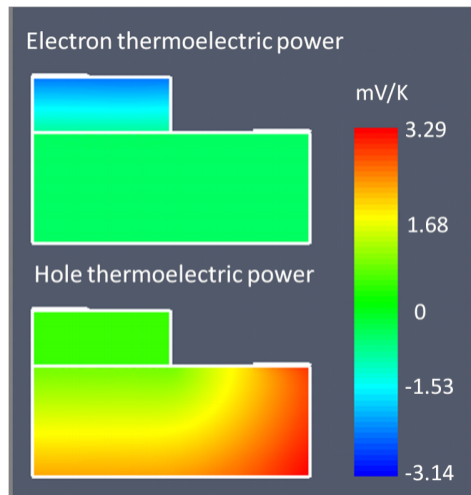
```
HeatSource constant{ H=1e6 }
```

The heat source must be in W/cm^3 . Heat generated by the Joule's effect is included with *heat_source joule* . The name of the drift-diffusion simulation is given by *dd_simulation* .



Example:

```
HeatSource joule{ dd_simulation = dd }
```



QUANTUM EFA CALCULATIONS

In TiberCAD, it is possible to perform quantum calculations in the framework of Envelope Function Approximation (EFA): eigenstates and eigenfunctions of a given system, dispersion of quantum states and particle quantum density can be obtained respectively by means of the following Modules:

- Module efaschroedinger
- Module quantumdispersion
- Module quantumdensity

The optical properties are calculated by the following modules

- Module opticskp
- Module opticalspectrum

11.1 Module efaschroedinger

The efaschroedinger simulation tool of TIBERCAD is developed in order to solve a single-particle Schroedinger equation for electrons and holes in a semi- conductor crystal. This problem is an eigenvalue problem that is treated as a generalized complex eigenvalue problem

$$H\psi = ES\psi, \quad (11.1)$$

where H and S are the Hamiltonian and S-matrix, respectively. The EFA calculations are performed by the **Module** efaschroedinger A typical ex- ample is the following:

```
Module efaschroedinger
{
    particle = el
    poisson_model_name = dd
    strain_model_name = strain # macrostrain
    name = quantum_el
}
```

```
regions = quantum
plot = (EigenFunctions, EigenEnergy, EnergyLevels)
Solver
{
  number_of_eigenstates = 10 # 30
}
Physics
{
  model = conduction_band
}
}
```

11.1.1 Module options

The following options influence the behaviour of the Module efaschroedinger:

`particle = string` defines for which particle (electron or hole) Schroedinger equation is

solved Possible values are el and hl. A different Module efaschroedinger has to be defined for each particle to be solved.

`poisson_model_name = string` defines the name of the simulation (Module driftdiffu-

sion) that can provide electric potential

`strain_model_name = string` defines the name of the simulation (Module macrostrain)

that can provide elastic strain

`regions = string` defines the regions associated to this EFA simulation

11.1.2 Solver section

The Solver section of the Module efaschroedinger contains the following options:

`number_of_eigenstates = integer` defines the number of eigenvalues and eigenfunctions to be found.

`Dirichlet_bc_everywhere = boolean` : if true (default value), Dirichlet boundary

conditions are imposed over all the boundaries of the simulation region

`solver = string` : defines the solver for the eigenvalue problem, possible values are:

arnoldi, lapack, krylovshur. The default value is **krylovshur**.

In the case of the lapack solver all the eigenvalues are computed. In the case of arnoldi or krylovshur solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, referred to as the *spectral_shift*

`max_iteration_number = integer`: maximum number of iteration, used as a stop condition

`eigen_solver_tolerance = double`: numerical eigensolver tolerance used as a convergence criteria

`spectral_shift = double`: the closest eigenvalues to this value (eV) are found. If not

defined, then it will be calculated internally from the band edges.

`ksp_type = string`: Krylov subspace method type; bcgsl, gmres, cg

`pc_type = string`: preconditioner type: cholesky, jacobi, ilu, composite.

11.1.3 Physics section

`model = string`: possible values are : conduction band, for single conduction band

`model (Γ point)`; kp for $\mathbf{k} \cdot \mathbf{p}$ model

`kp_model = string`: possible values are : 6?6, 8?8.

11.1.4 Output

The available output variables, to be specified in the plot option, are the following:

- EigenEnergy Eigen energy in eV
- EigenFunctions $|\psi(\mathbf{r})|^2$ function of the eigenstate
- Occupation probability to find the state occupied. It is calculated assuming Fermi

distribution and mean electrochemical potential and temperature:

$$\bar{\mu} = \langle \psi | \mu(\mathbf{r}) | \psi \rangle \bar{T} = \langle \psi | T(\mathbf{r}) | \psi \rangle \quad (11.2)$$

- EnergyLevels graphical output used for showing the energy level over the band diagram.

11.2 Module quantumdispersion

With the Module quantumdispersion it is possible to calculate the dependence of quantum eigenstates on **k**-vector. Such dependence gives the *quantum state dispersion*. The simulation name is **quantumdispersion**.

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  output_format = grace
  plot = k-space_dispersion
}
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

- **quantum simulation** : name of the efascroedinger simulation.
- **min eigenvalue number** , **max eigenvalue number** : the dispersion is calculated

for the states number i , where

$$\text{max_eigenvalue_number} \geq i \geq \text{min_eigenvalue_number}$$

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

11.2.1 Output

The output variable name is **k-space_dispersion**. The output format for the dispersion can be controlled independently of the general specification in the Simulation section by redefining the `output_format` keyword.

11.3 Module quantumdensity

In Module quantumdensity you can define the calculation of particle (electron,hole) density, based on the result of a previous calculation of the system eigenstates (e.g. with efascroedinger module). Quantum density may be obtained with an analytical or a numerical calculation.

Analytical calculation of density is done in the following way. For each eigenstate we calculate the effective mass assuming quadratic dispersion. Then the charge density is calculated as follows:

$$\rho_{1D}(\mathbf{r}) = g \frac{mkT}{2\pi\hbar^2} |\psi(\mathbf{r})|^2 \ln \left(1 + \exp \left(\pm \frac{\mu - E}{kT} \right) \right) \quad \rho_{2D}(\mathbf{r}) = g |\psi(\mathbf{r})|^2 \frac{1}{2} \sqrt{\left(\frac{mkT}{2\pi\hbar^2} \right)} F_{-1/2} \left(\pm \frac{\mu - E}{kT} \right), \quad (11.3)$$

where ρ_{1D} and ρ_{2D} are the 1D and 2D charge densities; m is the averaged mass (the mass is different for each quantized state and is position independent); g is the degeneracy of the states. The + sign is for electrons, the - sign is for holes.

Numerical calculation is done by the following formula:

The integration is performed on a mesh in the k-space.

```
Module quantumdensity
{
  name = dens_el
  regions = quantum
  plot = quantum_density
  k-space_dimension = 0
  k-space_basis = true
  k1 = (0, 0, 0.1)
  #number_of_nodes = (4)
  #wedge = half
  quantum_simulation = quantum_el
  degeneracy = 2
  refine_fraction = 0.20
  relative_accuracy = 0.01
  refine_k_space = true
  output_density_in_k_space = true
  uniform_refinement = false
  mesh_order = FIRST
  first_state = 3
  initial_eigenstates_number = 10
  analytic = true
}
```

The available options are:

- **quantum_simulation** : name of the efashroedinger simulation.
- degeneracy: degeneracy of the quantum state
- **initial_eigenstates_number** : initial number of eigenstates for the Schroedinger

equation

- **analytic** = { true | false } If true then the density is calculated analytically,

otherwise numerically.

11.3.1 Output

The output parameter is **quantum_density**.

11.4 Module **opticskp**

By defining the Module **opticskp**, calculation of optical properties is enabled; in particular, the optical kp matrix elements are calculated from the quantum models specified in the Module.

The optical spectrum from spontaneous emission is calculated in the following way:

where f_i and f_j are the Fermi distributions.

```
Module opticskp
{
  name = optics
  regions = quantum
  plot = (optical_spectrum_k_0 )
  initial_state_model = quantum_el
  final_state_model = quantum_hl
  initial_eigenstates = (0, 9)
  final_eigenstates = (0, 15)
  polarization = (0, 0, 1)
  Emin = 2.8
  Emax = 3.6
  dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

11.4.1 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for $k=0$.

11.5 Module opticalspectrum

By defining the Module **opticalspectrum** , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

`k_space_dimension` = **1** for 2D simulations, **2** for 1D simulations. k-space basis is

true if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in real space

If `refine_k_space` = **true** , that is adaptive k-mesh refinement is enabled, all the el-

ements whose error is greater than the value (**1-refine fraction**) (maximum error) are going to be refined. In this case, “Error” is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

`number_of_nodes` = numb. of elements in k mesh, along each direction

`wedge` = half | quarter, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin`, `Emax`, `dE` : energy range and step of spectrum calculation.

11.5.1 Output

The output variables for optics calculations are:

- **optical_spectrum** : k-space integrated optical emission spectrum.

SIMULATION DYE SOLAR CELLS

For a brief list of literature to understand Dye Solar cells (DSC) see [\[Kalyanasundaram\]](#) . For a review of the model see [\[Gagliardi\]](#) .

The model consists in a set of drift-diffusion equations for the description of the propagation of different ions and for the electrons coupled with Poisson equation:

$$\nabla \cdot (\mu_e n_e \nabla \phi_e) = (G - R) \quad (12.1)$$

$$\nabla \cdot (\mu_{I^-} n_{I^-} \nabla \phi_{I^-}) = -\frac{3}{2}(G - R) \quad (12.2)$$

$$\nabla \cdot (\mu_{I_3^-} n_{I_3^-} \nabla \phi_{I_3^-}) = \frac{1}{2}(G - R) \quad (12.3)$$

$$\nabla \cdot (\mu_c n_c \nabla \phi_c) = 0, \quad (12.4)$$

where μ_α refers to carrier mobilities, n_α to concentrations and ϕ_α to electrochemical potentials. R is the recombination term and G the generation term due to illumination. The Poisson equation to handle the internal potential drop:

$$-\varepsilon \Delta \varphi = q[n_c + N_D^+ - n_{I^-} - n_{I_3^-} - (n_e - \bar{n}_e)], \quad (12.5)$$

where N_D^+ is the amount of ionized dyes and it is equal to:

$$N_D^+ = \frac{G}{k_3} \quad (12.6)$$

with G the generation term and k_3 the rate constant of dye regeneration. The dielectric constant, ε , of the mesoporous material is a mix the two dielectric functions of the semi-conductor and the electrolyte. We use the Maxwell-Garnet model where the dielectric function of the mixed medium becomes:

$$\varepsilon = \varepsilon_s \frac{\varepsilon_e + 2\varepsilon_s + 2\varepsilon_p \varepsilon_e - 2\varepsilon_s \varepsilon_p}{\varepsilon_e + 2\varepsilon_s - \varepsilon_p \varepsilon_e + \varepsilon_s \varepsilon_p} \quad (12.7)$$

where ϵ_s and ϵ_e are the dielectric constants of the semiconductor and the electrolyte, respectively, and ϵ_p is the porosity of the medium. The recombination term depends largely on the loss mechanisms at the electrolyte/oxide interface which follows the reaction chain:



From the chemical path we can get a formula for the interface recombination (considering that the first chemical reaction is the slow process):

$$R = k_e \left[\left(\frac{n_e}{\bar{n}_e} \right)^\beta \bar{n}_e \sqrt{\frac{n_{I_3^-}}{n_{I^-}}} - \bar{n}_e \sqrt{\frac{\bar{n}_{I_3^-}}{\bar{n}_{I^-}^3}} n_{I^-} \right], \quad (12.11)$$

where the electron rate k_e is the recombination rate constant.

For the boundary conditions of the model we assume at the photoanode:

- $\phi_e = V$: electrochemical potential of electrons set with the voltage applied;
- $\nabla \phi_{I^-} = 0$: no iodide current at the photoanode;
- $\nabla \phi_{I_3^-} = 0$: no triiodide current at the photoanode;
- $\nabla \phi_c = 0$: no cationic current;
- $\nabla \varphi = 0$: no charged layer at the photoanode;

at the cathode:

- $\nabla \phi_e = 0$: no electronic current at the cathode;
- $-q\mu_{I^-} n_{I^-} \nabla \phi_{I^-} = \frac{3}{2} \left(\frac{-E_{red}(\mathbf{r}_c)}{R_L} \right)$: split of the current between the ionic species;
- $-q\mu_{I_3^-} n_{I_3^-} \nabla \phi_{I_3^-} = -\frac{1}{2} \left(\frac{-E_{red}(\mathbf{r}_c)}{R_L} \right)$: split of the current between the ionic species;
- $\nabla \phi_c = 0$: no cationic current;

integral boundary conditions for conservation of ionic species:

- $\int_{\Omega} \left[\frac{1}{3} n_{I^-}(\mathbf{r}) + n_{I_3^-}(\mathbf{r}) \right] d\mathbf{r} = \left(\frac{1}{3} \bar{n}_{I^-} + \bar{n}_{I_3^-} \right) \Omega$: conservation of iodine ions within the cell;
- $\int_{\Omega} n_c(\mathbf{r}) d\mathbf{r} = \bar{n}_c \Omega$: conservation of cation within the cell;

where ω is the volume of the cell, n_α the density of charged species and the index α stands for cation (c), iodide (I^-), triiodide (I_3^-) and electrons (e). R_L is the external load. The bias applied is equal to:

$$V = \phi_e - E_{red}. \quad (12.12)$$

E_{red} is the redox potential. The redox potential can be evaluated using a Nernst approximation:

$$E_{red} = E_{Pt}^0 - \frac{kT}{2} \ln \left(\frac{n_{I_3^-}/n_{St}}{(n_{I^-}/n_{St})^3} \right). \quad (12.13)$$

12.1 Module DSC

The DSC module is tagged as dssc. In this part of the input file are set the name of the simulation and the list of plotted variables:

```
Module dssc
{
  name = dssc
  plot = (Potential, Density, Current, ContactCurrents)

  Solver linesearch
  {
    max_iterations = 300
    step_tolerance = 1e-4

    linear_solver
    {
      preconditioner = lu
    }
  }

  Physics
  {
    ...
  }

  Contact anode
  {
    ...
  }

  Contact cathode
  {
    ...
  }
}
```

This section contains information about the **Contacts** , parameters for the **Solver** and the **Physics** sections.

12.1.1 Contacts

Information for the contacts is inserted in the **Module** section, in the subsections **Contacts** . The two contacts are the photoanode and the cathode. The photoanode must be the boundary of a region where TiO_2 is present, on the contrary the cathode must be the boundary of a region where the **electrolyte** is present.

```
Contact anode
{
  type = ohmic
  bias = $V[0.0]
}
```

```
Contact cathode
{
  type = Pt
  load = $R[1e8]
}
```

The anode is an ohmic contact, it contains only the sweep of the bias applied for the electrochemical potential of the electrons. The cathode must contain the initial external resistance (`load = R`). If a simulation of the cell under illumination is made the initial value of load must be set to a high value, in the range of $10^6 - 10^8 \Omega cm^2$ in order to have no current during the light intensity sweep. In case a simulation under dark conditions is performed, where an external bias only is applied, `load = 1.0`.

12.1.2 Physics

For the generation:

```
generation = dssc_generation
```

The **Physics** section must contain at least the setting of the generation module. The set of parameters that can be defined for the entire device are enlisted in table *below* .

12.2 Generation module

The generation term is related to the flux of photons which reaches the active TiO_2 regions and the dye present in the cell. We assume a simple Beer-Lambert exponential decay for charge generation of the form:

$$G(\mathbf{r}) = \int_{\lambda_1}^{\lambda_2} \alpha(\lambda) \Phi(\lambda) e^{-\alpha(\lambda) \hat{n} \cdot \mathbf{r}} d\lambda \quad (12.14)$$

Parameters		
Poisson equation		
porosity	Porosity of porous material	
perm_oxide	Relative perm. of the TiO ₂	
perm_electrolyte	Relative perm. of the electrolyte	
k_3	Oxidized Dye regeneration rate constant	s ⁻¹
Drift Diffusion equation		
ne	Dark electron density	cm ⁻³
nI	Dark iodide density	mol/l
nI3	Dark electron density	mol/l
mu_e	Electron mobility	cm ² /Vs
D_I	Iodide diffusion constant	cm ² /s
D_I3	Triiodide diffusion constant	cm ² /s
D_C	Cation diffusion constant	cm ² /s
Recombination term		
k_e	Recombination constant rate	s ⁻¹
beta	Non-linearity exponent in the electron density recombination	

Table 12.1: Physical parameters that can be set for the model divided in subsets relative to different processes in the cell.

where α is the absorption coefficient (in μm^{-1}) of the chosen Dye, $\Phi(\lambda)$ the intensity of the light power at wavelength λ of the light source spectrum.

The part of the input file for the generation module:

```
Module dssc_generation
{
  regions = TiO2
  plot = (Distance, Generation)
  light_direction = (1, 0, 0)
  light_intensity = $x
  dye = N719

  Contact anode
  {
  }
}
```

In the generation module must be specified:

- (regions) the regions where we want that there is generation (where the Dye is present);
- (plot) if we want to plot the generation;
- (light_direction) the vector which fixes the direction from where the light comes;

- (`light_intensity`) the light intensity;
- (`dye`) the dye used in the cell;
- (`illumination_spectrum`) the source spectrum, by default a 1.5 AM solar spectrum;

The light intensity is defined in a sweep. It can be set to reach 1 that means one Sun, or a larger or smaller illumination intensity (0.1, 2.0, etc.).

There is another flag called **illumination_spectrum** that can be used if it is wanted to change the spectrum profile F with another illumination source (for example if it is assumed that the cell is under light concentration). The file used by default for Φ is the standard 1.5 AM sun spectrum contained in the material database in the file `Sun1p5am`. The last part of the generation is the definition of the Contact. This Contact tells to the code which is the boundary region of the grid from where the light penetrates into the device. The information given by the boundary and the light direction vector defines the coming of light and direction of rays.

12.3 Device

Device section for a DSC simulation:

```
# Description of the device physical regions Device
{
  meshfile = <name_of_the_mesh>

  Region TiO2
  {
    material = TiO2mes
    porosity = 0.5
  }

  Region electrolyte
  {
    material = TiO2mes
    TiO2 = false
  }
}
```

The Device section must contain the name of the mesh file

```
meshfile = <name_of_the_mesh>.
```

For every region of the device it is specified the material file from the database (the **TiO2mes** contains standard parameters for both TiO2 and electrolyte, so it can be used for both regions).

For every region must be specified if it contains TiO2, or electrolyte or both. This can be done setting two flags called `TiO2` and `electrolyte`. If set **true** the material is present in the region, if set **false** it is not present. By default they are assumed both **true** (porous region). In the

second region of the example showed here there is electrolyte only and **TiO2 = false** is explicitly specified. In case both materials are present (porous region) a porosity must be specified (in the range between 0, TiO2 only, and 1, electrolyte only). If one material is not present the porosity is automatically set to 0 or 1.

12.4 Sweep

Three sweeps are needed for the plot of the entire I-V under illumination. The first sweep is needed to make the transition from dark condition to full open-circuit condition under illumination. The high value of the load R maintains the current to zero. Then a second sweep is used to pass from open circuit condition to short circuit condition lowering the external load from a high external load to a small one ($R = 1$).

Finally, the voltage sweep compute the I-V characteristics under illumination. In case of dark simulation (application of an external bias without illumination) the first and second sweep are not needed. The external load for the cathode can be set directly to $R = 1$.

Module sweep

```
{
  name = sweep_gen
  solve = (dssc_generation, dssc)
  variable = $x
  values = (0, 1e-9, 1e-8, 1e-7, 1e-6,
            1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)

  plot_data = true
}
```

Module sweep

```
{
  name = sweep_R
  solve = dssc
  variable = $R
  values = ( 1000, 100, 10, 1 )
  plot_data = true
}
```

Module sweep

```
{
  name = sweep_V
  solve = dssc
  variable = $V
  values = (0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.62, 0.64, 0.66,
            0.68, 0.7, 0.72, 0.74, 0.76, 0.78, 0.8)
  plot_data = true
}
```

}

The intensity of illumination can be changed sweeping the value of x different to 1 (1 = Sun of power).

12.5 Output

The output that want to be plotted are enlisted in section `Module dssc`.

See tables:

Nodal quantities		
	Potential	
eQFermi	Electron electrochemical potential	$\text{eV} (-e\phi_n)$
IQFermi	Iodide electrochemical potential	$\text{eV} (-e\phi_{I^-})$
I3QFermi	Triiodide electrochemical potential	$\text{eV} (-e\phi_{I_3^-})$
CQFermi	Cation electrochemical potential	$\text{eV} (-e\phi_C)$
ElPotential	Electrostatic potential	eV
Eredox	Electrolyte electrochemical potential	$\text{eV} (-e\phi_{Red})$
	Density	
eDensity	Electron density	cm^{-3}
IDensity	Iodide density	cm^{-3}
I3Density	Triiodide density	cm^{-3}
CDensity	Cation density	cm^{-3}
Generation	The net electron generation rate	$\text{cm}^{-3}\text{s}^{-1}$
NetRecombination	The net recombination rate	$\text{cm}^{-3}\text{s}^{-1}$

Table 12.2: Nodal quantities. The flags `Potential` and `Density` allow to plot all the electrochemical potentials and all the densities, including generation and recombination.

Elemental quantities		
	Current	
ElField	Electric Field	Vcm^{-1}
CurrentDensity	Total current density	Acm^{-2}
eCurrentDensity	Electron current density	Acm^{-2}
ICurrentDensity	Iodide current density	Acm^{-2}
I3CurrentDensity	Triiodide current density	Acm^{-2}
CCurrentDensity	Cation current density	Acm^{-2}

Table 12.3: Elemental quantities. The flag `Current` allows to plot all of them in the x , y and z components.

Scalar quantities

ContactCurrents	Contact currents	* ^a
-----------------	------------------	----------------

^adepends on dimension and symmetry

Table 12.4: Scalar quantities.

Part III

Reference Guide

ELASTICITY

Stiffness/Isotropic

Kind: bulk

Name: Stiffness

Type: isotropic

database_section: none

Options:

young (double,0.0)

poisson (double [-0.5,1.0],0.0)

Stiffness/Anisotropic

Kind: bulk

Name: Stiffness

Type: anisotropic

database_section = stiffness

Options:

none

Body Force/constant

Kind: bulk

Name: BodyForce

Type: constant

database_section: none

Options:

force (double vector,(0.0,0.0,0.0))

Body Force/Lattice Mismatch

Kind: bulk

Name: BodyForce

Type: lattice_mismatch

database_section: none

Options:

x (double [0.0,1,0],0.0)

x-growth-direction (double vector)

y-growth-direction (double vector)

z-growth-direction (double vector)

Body Force/Converse

Kind: bulk

Name: BodyForce

Type: converse

database_section: piezoelectricity

Options:

poisson_simulation (string,"none")

Boundary/clamp

Kind: interface

Name: Contact

Type: converse

database_section: none

Options:

poisson_simulation (string,"none")

Boundary/Surface Force

Kind: interface

Name: Contact

Type: surface_force

database_section: none

Options:

force (double vector,(0.0,0.0,0.0))

SOLVERS

14.1 Numerical Solvers

the “Solver” block inside a module description contains the options for the numerical solver. The solver type the “Solver” block is describing (linear, nonlinear, eigenvalue solver) depends on the module. For nonlinear and linear solvers, the following options exist:

14.1.1 Nonlinear solvers:

type :

petsc = uses the PETSc nonlinear solver (SNES) (default)

linesearch = uses a linear linesearch implemented in TiberCAD

Nonlinear solvers are based on iterative methods, solving in each iteration a linear system. The linear solver used for this can be controlled by providing a block with keyword “linear_solver” containing the options for the linear solver (see linear solvers).

petsc

- *relative_tolerance* = convergence criterion based on relative residual l₂-norm
- *absolute_tolerance* = convergence criterion based on the l₂-norm of the residual
- *max_iterations* = maximum number of iterations
- *step_tolerance* = tolerance criterion based on the l₂-norm of the correction step
- *max_step* = maximum linesearch step (l₂-norm)
- “*divergence_tolerance* = divergence criterion

linesearch

- *absolute_tolerance* = convergence criterion based on the l₂-norm of the residual
- *step_tolerance* = tolerance criterion based on the linfinity-norm of the correction step

- `max_iterations` = maximum number of iterations

14.1.2 Linear solvers

`type = petsc`

Petsc

`method`, The Krylov subspace method to be used:

`bcgs = BiCGSTAB` `bcgsl = gmres = Generalized Minimal Residual` `bicg = Bi-Conjugate Gradient` `cg = Conjugate Gradient` `cgs = Conjugate Gradient Squared`
`richardson = Richardson` `pconly = only apply preconditioner`

- `relative_tolerance` = convergence criterion based on relative residual l_2 -norm
- `absolute_tolerance` = convergence criterion based on the l_2 -norm of the residual
- `max_iterations` = maximum number of iterations

Preconditioner :

`lu = LU` `ilu = incomplete LU` `jacobi = Jacobi` `cholesky = Cholesky` `none = no preconditioning` `composite = use a combination of Jacobi and iLU`

14.2 Simulations

Sweep

Performs a linear sweep for a given variable.

Options:

`solve` = a list of simulations to be solved, eg. (strain, dd)

`variable` = the sweep variable, eg. \$Vd

`start` = the first value of the sweep

`stop` = the last value of the sweep

`steps` = the number of steps the interval (start, stop) gets subdivided in

`values` = instead of start, stop and steps, provide the sweep values explicitly

`plot_data` = set to true if you want to plot after each step the simulation results (default is false)

`min_step` = the minimum absolute step size

`max_step` = the maximum absolute step size

`initial_step` = the absolute initial step size

`min_relative_step` = minimum relative step size, default is 1e-3

`max_relative_step` = maximum relative step size, default is 1

`initial_relative_step` = initial relative step size, default is 1

The relative step sizes refer to each single sweep step. If `max_relative_step` is less than one, each sweep step will be subdivided in smaller steps. If a simulation fails, the step gets reduced by half until the simulation succeeds, or until the minimum relative or absolute step size is reached. In the latter case, the sweep is assumed to have failed. As for any module, a name can be given to a sweep block. This is important when several sweeps are defined, and in particular when nested sweep (solve option of one sweep refers to another sweep) are used.

Example:

```
Module sweep
{
  solve = (dd, thermal)
  start = 0
  stop = 1
  steps = 10
  plot_data = true
}
```

14.3 Selfconsistent

Solves models in an iterative way to obtain a selfconsistent solution, optionally using a relaxation approach. options:

- `max_iteration` = the maximum number of iterations. Currently, when the maximum number of iterations is reached, the program only issues a warning and proceeds.
- `relative_tolerance` = the relative convergence tolerance for the observed variable in terms of the l2-norm
- `absolute_tolerance` = the absolute convergence tolerance for the observed variable in terms of the maximum-norm
- `relaxation_factor` = an optional relaxation factor (default is 1) to be applied to the observed variable
- `solve` = the simulations to be solved

Currently, the observed variable on which convergence control and relaxation is done is the system variable of the last simulation specified in 'solve'.

THERMAL

Heat Source/constant

Kind: bulk

Name: HeatSource

Type: constant

database_section: none

Options:

H (double,0.0)

Heat Source/joule

Kind: bulk

Name: HeatSource

Type: joule

database_section: none

Options:

dd_simulation (string,"driftdiffusion")

Boundary/Heat reservoir

Kind: interface

Name: Contact

Type: heat_reservoir

database_section: none

Options:

temperature (positive double,300)

Boundary/Surface Thermal Resistance

Kind: interface

Name: Contact

Type: surface_resistance

database_section: none

Options:

r_surf (positive double 0.0)

temperature (positive double (K),300)

Boundary/Flux

Kind: interface

Name: Contact

Type: flux

database_section: none

Options:

Flux (double vector (W/cm2, (0.0,0.0,0.0))

QUANTUM EFA CALCULATIONS

In TiberCAD, it is possible to perform quantum calculations in the framework of Envelope Function Approximation (EFA): eigenstates and eigenfunctions of a given system, dispersion of quantum states and particle quantum density can be obtained respectively by means of the following Modules:

- Module `efaschroedinger`
- Module `quantumdispersion`
- Module `quantumdensity`

The optical properties are calculated by the following modules

- Module `opticskp`
- Module `opticalspectrum`

16.1 Module `efaschroedinger`

The EFA calculation of eigenstates and eigenfunctions are performed by the **Module** `efaschroedinger`. A typical example is the following:

```
Module efaschroedinger
{
  particle = el
  poisson_model_name = dd
  strain_model_name = strain # macrostrain
  name = quantum_el
  regions = quantum
  plot = (EigenFunctions, EigenEnergy, EnergyLevels)
  Solver
  {
    number_of_eigenstates = 10 # 30
  }
}
```

```
Physics
{
  model = conduction_band
}
}
```

16.1.1 Module options

The following options influence the behaviour of the Module `efaschroedinger`:

`particle = string` defines for which particle (electron or hole) Schroedinger equation is

solved Possible values are `el` and `hl`. A different Module `efaschroedinger` has to be defined for each particle to be solved.

`poisson_model_name = string` defines the name of the simulation (Module `driftdiffu-`

sion) that can provide electric potential

`strain_model_name = string` defines the name of the simulation (Module `macrostrain`)

that can provide elastic strain

`regions = string` defines the regions associated to this EFA simulation

16.1.2 Solver section

The Solver section of the Module `efaschroedinger` contains the following options:

`number_of_eigenstates = integer` defines the number of eigenvalues and eigenfunctions to be found.

`Dirichlet_bc_everywhere = boolean` : if true (default value), Dirichlet boundary

conditions are imposed over all the boundaries of the simulation region

`solver = string` : defines the solver for the eigenvalue problem, possible values are:

`arnoldi`, `lapack`, `krylovshur`. The default value is **`krylovshur`** . In the case of the `lapack` solver all the eigenvalues are computed. In the case of `arnoldi` or `krylovshur` solver it is necessary to specify which and how many eigenvalues have to be computed. The idea is that the iterative solver calculates several eigenvalues that are close to a specific number, referred to as the *spectral_shift*

`max_iteration_number = integer` : maximum number of iteration, used as a stop condition

`eigen_solver_tolerance = double` : numerical eigensolver tolerance used as a convergence criteria

`spectral_shift = double` : the closest eigenvalues to this value (eV) are found. If not

defined, then it will be calculated internally from the band edges.

`ksp_type = string` : Krylov subspace method type; bcgsl, gmres, cg

`pc_type = string` : preconditioner type: cholesky, jacobi, ilu , composite.

16.1.3 Physics section

`model = string` : possible values are : conduction band , for single conduction band

`model (  $\Gamma$  point ) ; kp for  $\mathbf{k} \cdot \mathbf{p}$  model`

`kp_model = string` : possible values are : 6?6 , 8?8.

16.1.4 Output

The available output variables, to be specified in the plot option, are the following:

- `EigenEnergy` Eigen energy in eV
- `EigenFunctions` $|\psi(\mathbf{r})|^2$ function of the eigenstate
- `Occupation probability` to find the state occupied. It is calculated assuming Fermi

distribution and mean electrochemical potential and temperature:

- `EnergyLevels` graphical output used for showing the energy level over the band

diagram.

<<<<<<< .mine By defining the Module **opticskp** , calculation of optical properties is enabled; in particular, the optical kp matrix elements are calculated from the quantum models specified in the Module.

The optical spectrum from spontaneous emission is calculated in the following way:

where f_i and f_j are the Fermi distributions.

```
Module opticskp
{
  name = optics
```

```
regions = quantum
plot = (optical_spectrum_k_0 )
initial_state_model = quantum_el
final_state_model = quantum_hl
initial_eigenstates = (0, 9)
final_eigenstates = (0, 15)
polarization = (0, 0, 1)
Emin = 2.8
Emax = 3.6
dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole). *initial_eigenstates* and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

16.2 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for $k=0$.

MODULE OPTICALSPECTRUM

By defining the Module **opticalspectrum**, optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

`k_space_dimension` = **1** for 2D simulations, **2** for 1D simulations. `k-space basis` is

true if the k-space is defined by means of k-vectors; if **false**, vectors are expressed in real space

If `refine_k_space` = **true**, that is adaptive k-mesh refinement is enabled, all the elements whose error is greater than the value $(1 - \text{refine fraction}) \times (\text{maximum error})$ are going to be refined. In this case, “Error” is just the integrated quantity. The refinement will end when the

relative_accuracy is obtained.

`number_of_nodes` = numb. of elements in k mesh, along each direction

`wedge` = half | quarter, to reduce calculation time, by exploiting symmetry.

`optical_matr_elem_model` = name of the *opticskp* model associated

`polarization` = light polarization (vector)

`Emin`, `Emax`, `dE` : energy range and step of spectrum calculation.

OUTPUT

Module quantumdispersion >>>>>> .r2404 _____

With the Module quantumdispersion it is possible to calculate the dependence of quantum eigenstates on **k**-vector. Such dependence gives the *quantum state dispersion*. The simulation name is **quantumdispersion**.

```
Module quantumdispersion
{
  simulation_name = dispersion1D_el
  regions = all
  quantum_simulation = quantum_el
  min_eigenvalue_number = 0
  max_eigenvalue_number = 5
  wedge = half
  k_space_dimension = 1
  k1 = (0, 0.1, 0)
  number_of_nodes = (10)
  output_format = grace
  plot = k-space_dispersion
}
```

The dispersion of quantum states is calculated at k-points that are nodes of the mesh in k-space.

The main parameters are:

- **quantum simulation** : name of the efasc Schroedinger simulation.
- **min eigenvalue number** , **max eigenvalue number** : the dispersion is calculated

for the states number i , where

$$\text{max_eigenvalue_number} \geq i \geq \text{min_eigenvalue_number}$$

The rest of the parameters (wedge, k space dimension, etc...) define the k-space.

The output variable name is **k-space_dispersion**. The output format for the dispersion can be controlled independently of the general specification in the Simulation section by redefining the output_format keyword.

18.1 Module quantumdensity

In Module quantumdensity you can define the calculation of particle (electron, hole) density, based on the result of a previous calculation of the system eigenstates (e.g. with efascroedinger module). Quantum density may be obtained with an analytical or a numerical calculation.

Analytical calculation of density is done in the following way. For each eigenstate we calculate the effective mass assuming quadratic dispersion. Then the charge density is calculate as follows:

where ρ_{1D} and ρ_{2D} are the 1D and 2D charge densities; m is the averaged mass (the mass is different for each quantized state and is position independent); g is the degeneracy of the states. The + sign is for electrons, the - sign is for holes.

Numerical calculation is done by the following formula:

The integration is performed on a mesh in the k-space.

```
Module quantumdensity
{
  name = dens_el
  regions = quantum
  plot = quantum_density
  k-space_dimension = 0
  k-space_basis = true
  k1 = (0, 0, 0.1)
  #number_of_nodes = (4)
  #wedge = half
  quantum_simulation = quantum_el
  degeneracy = 2
  refine_fraction = 0.20
  relative_accuracy = 0.01
  refine_k_space = true
  output_density_in_k_space = true
  uniform_refinement = false
  mesh_order = FIRST
  first_state = 3
  initial_eigenstates_number = 10
  analytic = true
}
```

The available options are:

- **quantum_simulation** : name of the efascroedinger simulation.
- **degeneracy**: degeneracy of the quantum state
- **initial_eigenstates_number** : initial number of eigenstates for the Schroedinger

equation

- **analytic** = { true | false } If true then the density is calculated analytically, otherwise numerically.

18.1.1 Output

The output parameter is **quantum_density**.

18.2 Module **opticskp**

By defining the Module **opticskp**, calculation of optical properties is enabled; in particular, the optical kp matrix elements are calculated from the quantum models specified in the Module.

The optical spectrum from spontaneous emission is calculated in the following way:

where f_i and f_j are the Fermi distributions.

```
Module opticskp
{
  name = optics
  regions = quantum
  plot = (optical_spectrum_k_0 )
  initial_state_model = quantum_el
  final_state_model = quantum_hl
  initial_eigenstates = (0, 9)
  final_eigenstates = (0, 15)
  polarization = (0, 0, 1)
  Emin = 2.8
  Emax = 3.6
  dE = 0.001
}
```

Here, *initial_state_model* and *final_state_model* are, respectively, the quantum simulations (**efaschroedinger** module) associated respectively to the initial state of optical transition (e.g. electron), and to the final state of optical transition (e.g. hole).

initial_eigenstates and *final_eigenstates* refer to the range of eigenstates to be taken in account for optical calculations.

By specifying a range of energy values in this way:

```
Emin = 3.0
Emax = 5.0
dE = 0.001
```

the emission optical spectrum for **k=0** is calculated.

18.2.1 Output

The output variables for optics calculations are:

- **optical_spectrum_k_0** : optical emission spectrum for $k=0$.

18.3 Module opticalspectrum

By defining the Module **opticalspectrum** , optical matrix elements are used to calculate the associated (emission) spectrum with a k-space integration.

```
Module opticalspectrum
{
  k_space_dimension = 2
  k-space_basis = true
  k1 = (0, 0, 0.1)
  k2 = (0, 0.1, 0)
  refine_fraction = 0.30
  relative_accuracy = 0.01
  refine_k_space = true
  number_of_nodes = (2, 2)
  wedge = quarter
  plot = (optical_spectrum)
  optical_matr_elem_model = opticskp
  polarization = (0, 0, 1)
  Emin = 3.0
  Emax = 5.0
  dE = 0.001
}
```

The parameters are the following:

k_space_dimension = **1** for 2D simulations, **2** for 1D simulations. **k-space basis** is

true if the k-space is defined by means of k-vectors; if **false** , vectors are expressed in real space

If **refine_k_space** = **true** , that is adaptive k-mesh refinement is enabled, all the elements whose error is greater than the value (1-refine fraction) (maximum error) are going to be refined. In this case, Error is just the integrated quantity. The refinement will end when the *relative_accuracy* is obtained.

number_of_nodes = numb. of elements in k mesh, along each direction

wedge = half | quarter, to reduce calculation time, by exploiting symmetry.

optical_matr_elem_model = name of the *opticskp* model associated

polarization = light polarization (vector)

Emin, Emax, dE : energy range and step of spectrum calculation.

18.3.1 Output

The output variables for optics calculations are:

- **optical_spectrum** : k-space integrated optical emission spectrum.

Part IV

Glossary

abs_tolerance

Level: paragraph

Description:

Example:

`abs_tolerance = 1e-3`

analytic

Level: paragraph

Description: { true or false } If true then the density is calculated analytically, otherwise numerically.

Example:

`analytic = true`

area_factor

Level: paragraph

Description:

Example:

`area_factor = 0.1`

barrier_height

Level: paragraph

Description:

Example:

`barrier_height = 3.0`

beta

Level: paragraph

Description:

Example:

beta = 1.0

bias

Level: paragraph

Description:

Example:

bias = @V[0.0]

Contact/anode

Level: Block

Description:

Example:

```
Contact anode
{
    type = ohmic
    voltage = $Vb[0.0]
}
```

Contact/backcontact

Level: Block

Description:

Example:

```
Contact backcontact
{
    voltage = 0.0
    area_factor = 0.1
}
```

Contact/Base

Level: Block

Description:

Example:

```
Contact Base
{
    type = clamp
}
```

Contact/cathode

Level: Block

Description:

Example:

```
Contact cathode
{
    type = ohmic
    voltage = 0.0
}
```

Contact/dissipator

Level: Block

Description:

Example:

```
Contact dissipator
{
    r_surf = 0.5
    type = surface_resistance
    temperature = 300
}
```

Contact/gate

Level: Block

Description:

Example:

```
Contact gate
{
    regions = (gate1, gate2)
    type = schottky
    work_function = 1.5272
    voltage = @Vg
}
```

Contact/heat_reservoir

Level: Block

Description:

Example:

```
Contact heat_reservoir
{
    region = substrate
    temperature = 300
}
```

Contact/left

Level: Block

Description:

Example:

```
Contact left
{
    type = heat_reservoir
    temperature = 300
}
```

Contact/reservoir

Level: Block

Description:

Example:

```
Contact reservoir
{
    temperature = 300
}
```

Contact/right

Level: Block

Description:

Example:

```
Contact right
{
    type = heat_reservoir
    temperature = 300
}
```

Contact/substrate

Level: Block

Description:

Example:

```
Contact substrate
{
    type = surface_resistance
    r_surf = 0. 05
    temperature = 300
}
```

Contact/surface_resistance

Level: Block

Description:

Example:

```
Contact surface_resistance
{
    r_surf = 0.01
    temperature = 300
}
```

Contact/Top

Level: Block

Description:

Example:

```
Contact Top
{
```

```
        type = surface_force
        force = (0.0, 0.0625,0.0)
    }
```

coupling

Level: paragraph

Description:

Example:

```
coupling = electrons
```

dd_simulation

Level: paragraph

Description: The name of the drift-diffusion simulation is given by dd_simulation.

Example:

```
dd_simulation = dd
```

dE

Level: paragraph

Description:

Example:

```
dE = 0.001
```

degeneracy

Level: paragraph

Description: degeneracy of the quantum state

Example:

degeneracy = 2

density

Level: paragraph

Description: doping concentration [cm-3]

Example:

density = 1e17

dimension

Level: paragraph

Description:

Example:

dimension = 3

Dirichlet_bc_everywhere

Level: paragraph

Description:

Example:

Dirichlet_bc_everywhere = true

discretization

Level: paragraph

Description:

Example:

discretization = fem

Doping

Level: sub-Block

Description:

Example:

```
Doping
{
    Nd = 1e16
    type = donor
}
```

doping_level

Level: paragraph

Description:

Example:

doping_level = 0.035

doping_type

Level: paragraph

Description:

Example:

doping_type = donors

dye

Level: paragraph

Description: the kind of dye used in the cell

Example:

dye = N719

E_{max}

Level: paragraph

Description:

Example:

E_{max} = 3.6

E_{min}

Level: paragraph

Description:

Example:

E_{min} = 2.8

final_eigenstates

Level: paragraph

Description: range of eigenstates to be taken in account for optical calculations.

Example:

final_eigenstates = (0, 15)

final_state_model

Level: paragraph

Description:

Example:

```
final_state_model = quantum_hl
```

first_state

Level: paragraph

Description:

Example:

```
first_state = 3
```

flavour

Level: paragraph

Description:

Example:

```
flavour = relaxation
```

force

Level: paragraph

Description:

Example:

```
force = (0.0, 0.0625,0.0)
```

generation

Level: paragraph

Description:

Example:

generation = dssc_generation

H

Level: paragraph

Description:

Example:

H = 1e10

initial_eigenstates

Level: paragraph

Description: range of eigenstates to be taken in account for optical calculations.

Example:

initial_eigenstates = (0, 9)

initial_eigenstates_number

Level: paragraph

Description: initial number of eigenstates for the Schroedinger equation

Example:

initial_eigenstates_number = 10

initial_state_model

Level: paragraph

Description:

Example:

```
initial_state_model = quantum_el
```

integration_order

Level: paragraph

Description:

Example:

```
integration_order = 2
```

k-space_basis

Level: paragraph

Description:

Example:

```
k-space_basis = true
```

k-space_dimension

Level: paragraph

Description:

Example:

```
k-space_dimension = 0
```

k1

Level: paragraph

Description:

Example:

$k1 = (0, 0, 0.1)$

k2

Level: paragraph

Description:

Example:

$k2 = (0, 0.1, 0)$

kp_model

Level: paragraph

Description:

Example:

$kp_model = 8 \times 8$

ksp_type

Level: paragraph

Description:

Example:

$ksp_type = ponly$ #preconditioner only

level

Level: paragraph

Description: energy level of the dopant [eV]

Example:

level = 0.025

light_direction

Level: paragraph

Description:

Example:

light_direction = <vector Indicating the direction of light>

light_intensity

Level: paragraph

Description: the vector which fixes the direction from where the light comes.

Example:

light_intensity = @x

load

Level: paragraph

Description:

Example:

load = @R[1e8]

low_field_model

Level: paragraph

Description:

Example:

low_field_model = doping_dependent

ls_max_step

Level: paragraph

Description:

Example:

ls_max_step = 4

mat

Level: paragraph

Description:

Example:

mat = TiO2mes

material

Level: paragraph

Description:

Example:

material = Si

max_eigenvalue_number

Level: paragraph

Description: the dispersion calculated for the states number i

Example:

`max_eigenvalue_number = 5`

max_iterations

Level: paragraph

Description:

Example:

`max_iterations = 5`

mesh_order

Level: paragraph

Description:

Example:

`mesh_order = FIRST`

mesh_regions

Level: paragraph

Description:

Example:

`mesh_regions = (barrier_dop_s, barrier_dop_d)`

mesh_units

Level: paragraph

Description:

Example:

`mesh_units = 1e-6`

meshfile

Level: Block

Description:

Example:

`meshfile = bulk.msh`

min_eigenvalue_number

Level: paragraph

Description: the dispersion calculated for the states number i

Example:

`min_eigenvalue_number = 0`

model

Level: paragraph

Description:

Example:

`model = conduction_band`

Modules/BodyForceconverse_piezo

Level: Block

Description:

Example:

```
BodyForceconverse_piezo
{
    poisson_simulation = dd
}
```

Modules/BodyForcelattice_mismatch

Level: Sub-Block

Description:

Example:

```
BodyForcelattice_mismatch
{
    x-growth-direction = (-1,0,1,0)
    y-growth-direction = (-1,2,-1,0)
    z-growth-direction = (0,0,0,1)
    reference_material = AlGaN
    structure = wz
    x = 0.2
}
```

Modules/Boundary

Level: Sub-Block

Description:

Example:

```
Boundary substrate
{
    regions = bottom
}
```

```
        material = GaN
        y-growth-direction = (0,0,0,1)
        z-growth-direction = (1,0,-1,0)
        x-growth-direction = (-1,2,-1,0)
    }
```

Modules/Cluster

Level: Block

Description:

Example:

```
Cluster semiconductor
{
    regions = -passivation
}
```

Modules/Device

Level: Super-Block

Description:

Example:

```
Device
{
    meshfile = bulk.msh
    Region bulk
    {
        material = Si
        Doping
        {
            Nd = 1e16
            type = donor
        }
    }
}
```

Modules/driftdiffusion

Level: Super-Block

Description:

Example:

```
Module driftdiffusion
{
    plot = (Ec, Ev, eQFermi, hQFermi, ContactCurrent)

    Contact anode { voltage = $Vb }
    Contact cathode { }
}
```

Modules/dssc

Level: Block

Description:

Example:

```
model dssc
{
    options
    {
        simulation_name = <name_of_the_model>
        plot(Potential, Density, Current, ContactCurrents)
    }
    ...
}
```

Modules/dssc_generation

Level: Block

Description:

Example:

```
model dssc_generation
{
  options
  {
    regions = (<TiO2_region_name_1>, <TiO2_region_name_2>, ...)
    plot = (Distance, Generation)
    light_direction = <vector Indicating the direction of light>
    (example, illumination from x direction: light_direction = (1, 0, 0))
    light_intensity = @x
    dye = N719
  }
  ...
}
```

Modules/efaschroedinger

Level: Block

Description:

Example:

```
Module efaschroedinger
{
  particle = el
  poisson_model_name = dd
  strain_model_name = strain # macrostrain
  name = quantum_el
  regions = quantum
  plot = (EigenFunctions, EigenEnergy, EnergyLevels)
  Solver
  { number_of_eigenstates = 10 # 30 }
  Physics
  { model = conduction_band }
```

Modules/elasticity

Level: Super-Block

Description:

Example:

```
Module elasticity
{
  Physics
  {
    Stiffness isotropic
    {
      Young = 129 Poisson = 0.349
    }
    Contact Base
    {
      type = clamp
    }
    Contact Top
    {
      type = surface_force force = (0.0, 0.0625,0.0)
    }
  }
}
```

Modules/HeatSource

Level: Sub-Block

Description:

Example:

```
HeatSource constant
{
  regions = HotSpot
  H = 1e10
}
```

Modules/LinearSolver

Level: Sub-Block

Description:

Example:

```
LinearSolver petsc
{
    tolerance = 1e-7
    max_iterations = 10000
    #xmonitor = true
    pc = composite
}
```

Modules/mobility

Level: Sub-Block

Description:

Example:

```
mobility field_dependent
{
    low_field_model = doping_dependent
}
```

Modules/NonlinearSolver

Level: Block

Description:

Example:

```
NonlinearSolver linesearch
{
    relative_tolerance = 1e-15
    step_tolerance = 5e-3
    max_iterations = 15
}
```

Modules/opticalspectrum

Level: Block

Description:

Example:

```
Module opticalspectrum
{
    k_space_dimension = 2
    k-space_basis = true
    k1 = (0, 0, 0.1)
    k2 = (0, 0.1, 0)
    refine_fraction = 0.30
    relative_accuracy = 0.01
    refine_k_space = true
    number_of_nodes = (2, 2)
    wedge = quarter
    plot = (optical_spectrum)
    optical_matr_elem_model = opticskp
    polarization = (0, 0, 1)
    Emin = 3.0
    Emax = 5.0
    dE = 0.001
}
```

Modules/opticskp

Level: Block

Description:

Example:

```
Module opticskp
{
    name = optics
    regions = quantum
}
```

```
    plot = (optical_spectrum_k_0 )
    initial_state_model = quantum_el
    final_state_model = quantum_hl
    initial_eigenstates = (0, 9)
    final_eigenstates = (0, 15)
    polarization = (0, 0, 1)
    Emin = 2.8
    Emax = 3.6
    dE = 0.001
}
```

Modules/particle_density

Level: Sub-Block

Description:

Example:

```
particle_density
{
    statistics = boltzmann
}
```

Modules/Physics

Level: Block

Description:

Example:

```
Physics
{
    particle_density
    {
        statistics = boltzmann
    }
    mobility field_dependent
    {
```



```
        low_field_model = doping_dependent
    }
    recombination (srh, auger) {}
}
```

Modules/quantumdensity

Level: Block

Description:

Example:

```
Module quantumdensity
{
    name = dens_el
    regions = quantum
    plot = quantum_density
    k-space_dimension = 0
    k-space_basis = true
    k1 = (0, 0, 0.1)
    quantum_simulation = quantum_el
    degeneracy = 2
    refine_fraction = 0.20
    relative_accuracy = 0.01
    refine_k_space = true
    output_density_in_k_space = true
    uniform_refinement = false
    mesh_order = FIRST
    first_state = 3
    initial_eigenstates_number = 10
    analytic = true
}
```

Modules/quantumdispersion

Level: Block

Description:

Example:

```
Module quantumdispersion
{
    simulation_name = dispersion1D_el
    regions = all
    quantum_simulation = quantum_el
    min_eigenvalue_number = 0
    max_eigenvalue_number = 5
    wedge = half
    k_space_dimension = 1
    k1 = (0, 0.1, 0)
    number_of_nodes = (10)
    output_format = grace
    plot = k-space_dispersion
}
```

Modules/Selfconsistent

Level: Block

Description:

Example:

```
Module Selfconsistent
{
    flavour = relaxation
    simulations = (dd,tt)
    abs_tolerance = 1e-5
    monitor = true
}
```

Modules/Solver

Level: Super-Block

Description:

Example:

```
Solver
{
    nonlinear_solver = tiber
    nonlin_max_it = 15
    nonlin_rel_tol = 1e-12
    nonlin_step_tol = 1e-5
}
```

Modules/Stiffness

Level: Block

Description:

Example:

```
Stiffness isotropic
{
    Young = 129
    Poisson = 0.349
}
```

Modules/sweep

Level: Super-Block

Description:

Example:

```
Module sweep
{
    solve = driftdiffusion
    variable = $Vb
    start = 0.0
    stop = 1
    steps = 10
    plot_data = true
}
```

Modules/sweep_gen

Level: Block

Description:

Example:

```
sweep_gen
{
    simulation = (dssc_generation, dssc)
    variable = x
    values = (0, 1e-9, 1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1)
    plot_data = true
}
```

Modules/trap

Level: Sub-Block

Description:

Example:

```
trap fixed_charge
{
    region = GaN/SiN
    Nt = 2.74e13
}
```

monitor

Level: paragraph

Description:

Example:

```
monitor = true
```

name

Level: paragraph

Description:

Example:

name = dd

Nd

Level: paragraph

Description:

Example:

Nd = 1e16

nonlin_abs_tol

Level: paragraph

Description:

Example:

nonlin_abs_tol = 1e-10

nonlin_max_it

Level: paragraph

Description:

Example:

nonlin_max_it = 15

nonlin_rel_tol

Level: paragraph

Description:

Example:

`nonlin_rel_tol = 1e-12`

nonlin_step_tol

Level: paragraph

Description:

Example:

`nonlin_step_tol = 1e-5`

nonlinear_solver

Level: paragraph

Description:

Example:

`nonlinear_solver = tiber`

number_of_eigenstates

Level: paragraph

Description: defines the number of eigenvalues and eigenfunctions to be found.

Example:

`number_of_eigenstates = 10 # 30`

number_of_nodes

Level: paragraph

Description: number of elements in k mesh, along each direction.

Example:

```
number_of_nodes = (10)
number_of_nodes = (2, 2)
```

optical_matr_elem_model

Level: paragraph

Description: name of the opticskp model associated

Example:

```
optical_matr_elem_model = opticskp
```

output_density_in_k_space

Level: paragraph

Description:

Example:

```
output_density_in_k_space = true
```

output_format

Level: paragraph

Description:

Example:

output_format = grace

particle

Level: paragraph

Description:

Example:

particle = el

plot

Level: paragraph

Description:

Example:

plot = (Ec, Ev, Eg, ElField, ElPotential, eQFermi, hQFermi, eDensity, hDensity,
NetRecombination, eCurrentDensity, hCurrentDensity, CurrentDensity,
ContactCurrent, eMobility, hMobility, IonizedDonors, IonizedAcceptors)
plot = all

plot_data

Level: paragraph

Description:

Example:

plot_data = true

Poisson

Level: paragraph

Description:

Example:

Poisson = 0.349

poisson_model_name

Level: paragraph

Description:

Example:

poisson_model_name = dd

poisson_simulation

Level: paragraph

Description:

Example:

poisson_simulation = dd

polarization

Level: paragraph

Description:

Example:

polarization (piezo, pyro) {}

polarization

Level: paragraph

Description:

Example:

polarization = (0, 0, 1)

porosity

Level: paragraph

Description:

Example:

porosity = 0.5

preconditioner

Level: paragraph

Description:

Example:

preconditioner = lu

quadrature_rule

Level: paragraph

Description:

Example:

quadrature_rule = trapez

quantum_simulation

Level: paragraph

Description: name of the efashroedinger simulation.

Example:

```
quantum_simulation = quantum_el
```

r_surf

Level: paragraph

Description:

Example:

```
r_surf = 0. 05
```

recombination

Level: paragraph

Description:

Example:

```
recombination (srh, auger) { }
```

reference

Level: paragraph

Description:

Example:

```
reference = cb
```

reference_material

Level: paragraph

Description:

Example:

reference_material = AlGaN

refine_fraction

Level: paragraph

Description:

Example:

refine_fraction = 0.20

refine_k_space

Level: paragraph

Description:

Example:

refine_k_space = true

region

Level: paragraph

Description:

Example:

region = GaN/SiN

region = buffer

Region/Air

Level: Block

Description:

Example:

```
Region Air
{
    material = Air
}
```

Region/barrier

Level: Block

Description:

Example:

```
Region barrier
{
    material = AlGaN
    x = 0.14
}
```

Region/barrier_doped

Level: Block

Description:

Example:

```
Region barrier_doped
{
    mesh_regions = (barrier_dop_s, barrier_dop_d)
    material = AlGaN
    x = 0.14
    Doping
    {
        density = 1e21
        type = donor
    }
}
```

```
        level = 0.026
    }
}
```

Region/buffer_doped

Level: Block

Description:

Example:

```
Region buffer_doped
{
    mesh_regions = (buffer_dop_s, buffer_dop_d)
Doping
{
    density = 1e21
    type = donor
    level = 0.026
}
}
```

Region/bulk

Level: Block

Description:

Example:

```
Region bulk
{
    material = Si
Doping
{
    Nd = 1e16
    type = donor
}
}
```

Region/cap

Level: Block

Description:

Example:

```
Region cap
{
  Doping
  {
    density = 5e18
    type = donor
    level = 0.026
  }
}
```

Region/cap_doped

Level: Block

Description:

Example:

```
Region cap_doped
{
  mesh_regions = (cap_dop_s, cap_dop_d)
  Doping
  {
    density = 1e21
    type = donor
    level = 0.026
  }
}
```

Region/collector

Level: Block

Description:

Example:

```
Region coll # the collector region
{
    type = donor
    Nd = 5e15
}
```

Region/electrolyte

Level: Block

Description:

Example:

```
Region <name of the electrolyte region>
{
    mat = TiO2mes
    TiO2 = false
}
```

Region/n_side

Level: Block

Description:

Example:

```
Region n_side
{
    Doping
    {
        Nd = 1e18
        type = donor
    }
}
```



```
}
```

Region/Nanowire

Level: Block

Description:

Example:

```
Region Nanowire
{
    doping_type = donors
    doping_level = 0.035
}
```

Region/oxide

Level: Block

Description:

Example:

```
Region oxide
{
    material = SiO2
}
```

Region/p_side

Level: Block

Description:

Example:

```
Region p_side
{
    Doping
```

```
{  
    Nd = 1e18  
    type = acceptor  
}  
}
```

Region/passivation

Level: Block

Description:

Example:

```
Region passivation  
{  
    mesh_regions = (passiv1, passiv2, passiv3, passiv4)  
    material = SiN  
}
```

Region/porous

Level: Block

Description:

Example:

```
Region <name of the porous region>  
{  
    mat = TiO2mes  
    porosity = 0.5  
}
```

Region/Well

Level: Block

Description:

Example:

```
Region Well
{
    material = GaN
}
```

regions

Level: paragraph

Description:

Example:

```
regions = all #default to not specify
regions = -passivation #apply to all regions except passivation
```

rel_tolerance

Level: paragraph

Description:

Example:

```
rel_tolerance = 1e-6
```

relative_accuracy

Level: paragraph

Description:

Example:

```
relative_accuracy = 0.01
```

relative_tolerance

Level: paragraph

Description:

Example:

relative_tolerance = 1e-15

resultpath

Level: paragraph

Description:

Example:

resultpath = output

save_state

Level: paragraph

Description:

Example:

save_state = true

searchpath

Level: paragraph

Description:

Example:

searchpath = ../../materials

simulation_name

Level: paragraph

Description:

Example:

```
simulation_name = dispersion1D_el
```

solution_method

Level: paragraph

Description:

Example:

```
solution_method = general
```

solve

Level: paragraph

Description:

Example:

```
solve = driftdiffusion  
solve = sweep
```

solver

Level: paragraph

Description:

Example:

```
solver = krylovshur
```

start

Level: paragraph

Description:

Example:

start = 0.0

statistics

Level: paragraph

Description:

Example:

statistics = boltzmann

statistics = FermiDirac

step_tolerance

Level: paragraph

Description:

Example:

step_tolerance = 5e-3

steps

Level: paragraph

Description:

Example:

steps = 10

stop

Level: paragraph

Description:

Example:

stop = 1

strain_model_name

Level: paragraph

Description:

Example:

strain_model_name = strain # macrostrain

strain_simulation

Level: paragraph

Description:

Example:

strain_simulation = macrostrain

strain_simulation = elasticity

structure

Level: paragraph

Description:

Example:

structure = wZ

symmetry

Level: paragraph

Description:

Example:

symmetry = cylindrical

temperature

Level: paragraph

Description:

Example:

temperature = 300

TiO2

Level: paragraph

Description:

Example:

TiO2 = false

type

Level: paragraph

Description:

Example:

type = donor

type = acceptor

type = field_dependent

type = heat_reservoir

type = surface_resistance
type = surface_force

uniform_refinement

Level: paragraph

Description:

Example:

uniform_refinement = false

values

Level: paragraph

Description:

Example:

values = (1000, 100, 1)

variable

Level: paragraph

Description:

Example:

variable = \$Vb

verbose

Level: paragraph

Description:

Example:

verbose = 2

wedge

Level: paragraph

Description: to reduce calculation time, by exploiting symmetry.

Example:

wedge = half

wedge = quarter

x

Level: paragraph

Description: the elementum percentage in the alloy

Example:

material = AlGaIn

x = 0.14

x-growth-direction

Level: paragraph

Description:

Example:

x-growth-direction = (0,0,0,1)

y-growth-direction

Level: paragraph

Description:

Example:

y-growth-direction = (1,0,-1,0)

Young

Level: paragraph

Description:

Example:

Young = 129

z-growth-direction

Level: paragraph

Description:

Example:

z-growth-direction = (-1,2,-1,0)

BIBLIOGRAPHY

- [Kalyanasundaram] Kalyanasundaram Kuppaswamy, “Dye-Sensitized Solar Cells”, *Editor Crc Pr Inc.* , 2010.
- [Gagliardi] Alessio Gagliardi, Matthias Auf der Maur, Desiree Gentilini and Aldo Di Carlo, “Modeling of Dye sensitized solar cells using a finite element method”, *J. Computational Electronics* , vol. 8, pp. 12, 2009.
- [Wachutka] Wachutka, IEEE Trans CAD Cicui. Integr and Syst. (1990)].
- [Povolotskiy] Povolotskiy et al. JAP (2006).
- [Wang] Wang et al. Science (2006).
- [Gao] Gao et al NanoLetter (2009).
- [Selberherr] Siegfried Selberherr, Analysis and Simulation of Semiconductor Devices, SpringerVerlag Wien New York, 1st edition, 1984.
- [Wachutka] Gerhard K. Wachutka, Rigorous thermodynamic treatment of heat generation and conduction in semiconductor device modeling,” IEEE Transaction on Computer-aided Design., vol. 9, pp. 11, 1990.

INDEX

A

abs_tolerance, 121
analytic, 121
Anisotropic
 Stiffness, 97
area_factor, 121

B

barrier_height, 121
beta, 121
bias, 122
Body Force
 constant, 97
 Converse, 98
 Lattice Mismatch, 98
Boundary
 clamp, 98
 Flux, 106
 Heat reservoir, 105
 Surface Force, 98
 Surface Thermal Resistance, 105

C

clamp
 Boundary, 98
constant
 Body Force, 97
 Heat Source, 105
Contact/anode, 122
Contact/backcontact, 122
Contact/Base, 123
Contact/cathode, 123
Contact/dissipator, 123
Contact/gate, 124
Contact/heat_reservoir, 124

Contact/left, 124
Contact/reservoir, 125
Contact/right, 125
Contact/substrate, 126
Contact/surface_resistance, 126
Contact/Top, 126
Converse
 Body Force, 98
coupling, 127

D

dd_simulation, 127
dE, 127
degeneracy, 127
density, 128
dimension, 128
Dirichlet_bc_everywhere, 128
discretization, 128
Doping, 129
doping_level, 129
doping_type, 129
DriftDiffusion
 Solution, 57
dye, 129

E

E_{max}, 130
E_{min}, 130

F

final_eigenstates, 130
final_state_model, 130
first_state, 131
flavour, 131
Flux

Boundary, 106
force, 131

G

generation, 131

H

H, 132

Heat reservoir

Boundary, 105

Heat Source

constant, 105

joule, 105

I

initial_eigenstates, 132

initial_eigenstates_number, 132

initial_state_model, 132

integration_order, 133

Isotropic

Stiffness, 97

J

joule

Heat Source, 105

K

k-space_basis, 133

k-space_dimension, 133

k1, 133

k2, 134

kp_model, 134

ksp_type, 134

L

Lattice Mismatch

Body Force, 98

level, 134

light_direction, 135

light_intensity, 135

linear

Solvers, 102

load, 135

low_field_model, 135

ls_max_step, 136

M

mat, 136

material, 136

max_eigenvalue_number, 136

max_iterations, 137

mesh_order, 137

mesh_regions, 137

mesh_units, 137

meshfile, 138

min_eigenvalue_number, 138

model, 138

Modules/BodyForceconverse_piezo, 138

Modules/BodyForcelattice_mismatch, 139

Modules/Boundary, 139

Modules/Cluster, 140

Modules/Device, 140

Modules/driftdiffusion, 141

Modules/dssc, 141

Modules/dssc_generation, 141

Modules/efaschroedinger, 142

Modules/elasticity, 142

Modules/HeatSource, 143

Modules/LinearSolver, 143

Modules/mobility, 144

Modules/NonlinearSolver, 144

Modules/opticalspectrum, 145

Modules/opticskp, 145

Modules/particle_density, 146

Modules/Physics, 146

Modules/quantumdensity, 147

Modules/quantumdispersion, 147

Modules/Selfconsistent, 148

Modules/Solver, 148

Modules/Stiffness, 149

Modules/sweep, 149

Modules/sweep_gen, 150

Modules/trap, 150

monitor, 150

N

name, 151

Nd, 151

nonlin_abs_tol, 151

nonlin_max_it, 151

nonlin_rel_tol, 152

nonlin_step_tol, 152
 nonlinear
 Solvers, 101
 nonlinear_solver, 152
 number_of_eigenstates, 152
 number_of_nodes, 153

O

optical_matr_elem_model, 153
 output_density_in_k_space, 153
 output_format, 153

P

particle, 154
 plot, 154
 plot_data, 154
 Poisson, 154
 poisson_model_name, 155
 poisson_simulation, 155
 polarization, 155
 porosity, 156
 preconditioner, 156

Q

quadrature_rule, 156
 quantum_simulation, 156

R

r_surf, 157
 recombination, 157
 reference, 157
 reference_material, 157
 refine_fraction, 158
 refine_k_space, 158
 region, 158
 Region/Air, 158
 Region/barrier, 159
 Region/barrier_doped, 159
 Region/buffer_doped, 160
 Region/bulk, 160
 Region/cap, 160
 Region/cap_doped, 161
 Region/collector, 161
 Region/electrolyte, 162
 Region/n_side, 162

Region/Nanowire, 163
 Region/oxide, 163
 Region/p_side, 163
 Region/passivation, 164
 Region/porous, 164
 Region/Well, 164
 regions, 165
 rel_tolerance, 165
 relative_accuracy, 165
 relative_tolerance, 165
 resultpath, 166

S

save_state, 166
 searchpath, 166
 selfconsistent
 Solvers, 103
 simulation_name, 166
 Simulations
 sweep, 102
 Solution
 DriftDiffusion, 57
 solution_method, 167
 solve, 167
 solver, 167
 Solvers
 linear, 102
 nonlinear, 101
 selfconsistent, 103
 start, 167
 statistics, 168
 step_tolerance, 168
 steps, 168
 Stiffness
 Anisotropic, 97
 Isotropic, 97
 stop, 168
 strain_model_name, 169
 strain_simulation, 169
 structure, 169
 Surface Force
 Boundary, 98
 Surface Thermal Resistance
 Boundary, 105
 sweep

Simulations, [102](#)

symmetry, [169](#)

T

temperature, [170](#)

TiO₂, [170](#)

type, [170](#)

U

uniform_refinement, [171](#)

V

values, [171](#)

variable, [171](#)

verbose, [171](#)

W

wedge, [172](#)

X

x, [172](#)

x-growth-direction, [172](#)

Y

y-growth-direction, [172](#)

Young, [173](#)

Z

z-growth-direction, [173](#)